Pengantar

Pengantar: Teknologi Web Modern

Deskripsi Mata Kuliah

Mata kuliah Teknologi Web memberikan pemahaman mendalam tentang ekosistem pengembangan web modern, mulai dari fondasi hingga teknologi terkini. Anda akan mempelajari arsitektur dasar web, peran HTML, CSS, dan JavaScript, serta menguasai framework populer seperti React dan Next.js. Pembelajaran akan difokuskan pada pendekatan Case-Based Learning (CBL), di mana setiap teori akan langsung diterapkan dalam studi kasus praktikum.

Kompetensi

Setelah menyelesaikan mata kuliah ini, Anda diharapkan memiliki kompetensi sebagai berikut:

- Mampu menjelaskan cara kerja teknologi-teknologi yang mendasari aplikasi web.
- Terampil dalam pengembangan web sisi klien (*front-end*) menggunakan HTML, CSS, dan JavaScript.
- Mampu menggunakan *framework* antarmuka pengguna (UI) modern untuk mempercepat pengembangan.
- Terampil dalam menggunakan *framework* web berbasis JavaScript seperti React atau Next.js untuk membangun aplikasi dinamis.
- Mampu membuat aplikasi web yang terintegrasi dengan data statis (JSON) dan API eksternal.

Pelaksanaan Perkuliahan

Perkuliahan akan berlangsung selama 16 pertemuan, dengan rincian sebagai berikut:

Pertemuan	Topik	Tugas
1	Pengenalan Web Development	Analisis Fungsionalitas Website Populer
2	HTML Dasar	Membuat Halaman Web Pribadi
3	CSS Dasar	Styling Halaman Web Pribadi
4	CSS Lanjutan (Flexbox & Grid)	Membuat Tampilan Layout Responsif
5	JavaScript Dasar	Membuat Kalkulator Sederhana
6	JavaScript & DOM	Membuat To-Do List Interaktif
7	Framework UI (Bootstrap/Tailwind)	Menerapkan Bootstrap/Tailwind pada Halaman Web
8	Ujian Tengah Semester (UTS)	Ujian Teori

9	Pengenalan React	Membuat Komponen React Sederhana
10	State Management & Hooks	Membuat Counter App dengan useState & useEffect
11	React Router & Navigasi	Membuat Single Page Application (SPA)
12	Pengenalan Next.js	Membuat Website Statis dengan Next.js
13	Server Components & Routing	Membangun Aplikasi Blog dengan Next.js
14	Form Handling & Otentikasi	Melindungi Rute Aplikasi Blog
15	Menggunakan API Eksternal	Membuat Direktori Film dengan API Publik
16	Ujian Akhir Semester (UAS)	Proyek Akhir & Ujian Teori

Pengumpulan Tugas

Seluruh tugas praktikum dan proyek akhir akan dikelola menggunakan **GitLab**. Mahasiswa diwajibkan untuk membuat satu repositori utama yang akan menjadi portofolio digital selama satu semester.

Struktur Repositori GitLab

Berikut adalah struktur yang disarankan untuk repositori `username/tekweb_2025` di GitLab. Struktur ini akan membantu mengorganisir tugas setiap pertemuan.

```
None

tekweb_2025/
├── .gitignore
├── README.md
├── index.html
└── pertemuan-2/
    └── index.html
```

Penjelasan Struktur:

- `tekweb_2025/`: Ini adalah folder utama atau repositori.
- `.gitignore`: File untuk mengabaikan file yang tidak perlu diunggah ke repositori.
- `README.md`: File yang menjelaskan tentang repositori ini.

- `index.html`: Ini adalah file utama yang berisi daftar tautan ke semua tugas yang dikumpulkan. File ini akan selalu diperbarui setiap kali ada tugas baru.
- `pertemuan-2/`: Sebuah folder untuk tugas dari Pertemuan 2. Semua file terkait tugas ini (misalnya `index.html`, gambar, dll.) disimpan di sini.

Pertemuan 1

Modul Pertemuan 1: Pengenalan Web Development

Durasi: 1 Pertemuan

Tujuan Pembelajaran

Setelah menyelesaikan modul ini, mahasiswa diharapkan mampu:

- **Menjelaskan** sejarah dan evolusi World Wide Web.
- **Memahami** arsitektur dasar *client-server* dan perannya dalam web.
- **Mengidentifikasi** secara konkret peran HTML, CSS, dan JavaScript.
- **Menggunakan** alat-alat pengembangan dasar (*text editor* dan *browser developer tools*).
- **Menganalisis** fungsionalitas dan teknologi yang digunakan pada website populer.

Materi: Fondasi Dasar Teknologi Web

Sejarah Singkat dan Evolusi Web

World Wide Web (WWW) diciptakan oleh Sir Tim Berners-Lee pada tahun 1989 di CERN. Awalnya, web hanyalah sistem untuk berbagi dokumen ilmiah statis melalui *hyperlink* (disebut **Web 1.0**). Seiring berjalannya waktu, web berevolusi menjadi platform yang lebih interaktif, di mana pengguna tidak hanya mengonsumsi tetapi juga menghasilkan konten (seperti blog, media sosial). Era ini dikenal sebagai **Web 2.0**. Saat ini, web terus berkembang menuju web semantik dan terdesentralisasi (**Web 3.0**).

> **Referensi Tambahan:** Untuk sejarah yang lebih mendalam, Anda bisa membaca artikel di MDN Web Docs: A brief history of the web.

Arsitektur Client-Server

Ini adalah model dasar bagaimana web bekerja. Bayangkan sebuah restoran:

- **Anda (Client)**: Mengajukan permintaan (*request*) kepada pelayan, misalnya, "Saya ingin menu."
- **Pelayan (Protokol HTTP)**: Mengantarkan permintaan Anda ke dapur.
- **Dapur (Server)**: Menyiapkan dan mengirimkan pesanan (*response*) kembali kepada Anda.

Dalam konteks web:

- **Client**: Perangkat Anda (komputer, *smartphone*) dan *browser* yang Anda gunakan (Chrome, Firefox).
- **Server**: Komputer bertenaga tinggi yang menyimpan semua file website dan data.
- **Protokol HTTP/HTTPS**: Aturan komunikasi yang digunakan oleh *client* dan *server* untuk saling bertukar data.

Ketika Anda mengetik `https://www.google.com` di browser, Anda (sebagai *client*) mengirimkan permintaan ke server Google. Server memproses permintaan tersebut dan mengirimkan kembali file HTML, CSS, dan JavaScript yang dibutuhkan untuk menampilkan halaman utama Google.

Tiga Pilar Utama: HTML, CSS, dan JavaScript

Untuk membangun sebuah website, kita menggunakan ketiga bahasa ini. Mereka bekerja sama layaknya kerangka, penampilan, dan fungsionalitas pada tubuh manusia.

- **HTML (HyperText Markup Language)**
    - **Peran**: Bertanggung jawab untuk **struktur konten** dari halaman web. HTML menentukan elemen-elemen seperti judul, paragraf, daftar, gambar, dan tautan. Tanpa HTML, browser tidak akan tahu bagaimana menyusun konten.
    - **Contoh Sederhana**: `<h1>Judul Halaman</h1>` untuk membuat judul, `<p>Ini adalah paragraf.</p>` untuk membuat teks.
- **CSS (Cascading Style Sheets)**
    - **Peran**: Bertanggung jawab untuk **tampilan visual**. CSS digunakan untuk mengatur warna, *font*, tata letak, dan responsivitas halaman. CSS memisahkan konten dari presentasi, membuat perubahan desain menjadi lebih mudah.
    - **Contoh Sederhana**: `h1 { color: blue; font-size: 24px; }` untuk membuat judul berwarna biru dengan ukuran 24px.
- **JavaScript**
    - **Peran**: Bertanggung jawab untuk **interaksi dan fungsionalitas dinamis**. JavaScript membuat halaman web menjadi "hidup". Contohnya adalah validasi formulir, animasi, atau memuat konten baru tanpa harus memuat ulang halaman.
    - **Contoh Sederhana**: Menggunakan JavaScript untuk membuat pesan muncul saat tombol diklik.

> **Referensi Tambahan**: Untuk pemahaman mendalam tentang ketiga bahasa ini, sumber terbaik adalah MDN Web Docs dan freeCodeCamp.

Alat-Alat Pengembangan

- **Text Editor**: Anda akan membutuhkan editor kode. Visual Studio Code (VS Code) sangat direkomendasikan karena ringan, gratis, dan memiliki banyak ekstensi yang membantu pengembangan.
- **Browser Developer Tools**: Fitur bawaan di browser (tekan `F12`) yang sangat penting. Anda bisa melihat struktur HTML, menginspeksi gaya CSS, dan menguji kode JavaScript di sana.

Studi Kasus: Analisis Fungsionalitas Website Populer

Setiap website yang kita kunjungi adalah studi kasus nyata dari penerapan HTML, CSS, dan JavaScript. Tugas ini akan membantu Anda melihat teori dalam praktik.

Tugas Praktikum

1. **Pilih Website**: Pilih satu website populer (contoh: situs berita, portal e-commerce) untuk dianalisis.
2. **Identifikasi Elemen**: Identifikasi elemen-elemen kunci seperti:
    - **Struktur (HTML)**: Menu navigasi, *header*, *footer*, artikel, atau daftar produk.
    - **Tampilan (CSS)**: Skema warna, tata letak kolom, atau efek visual pada tombol.
    - **Interaksi (JavaScript)**: Fitur seperti galeri gambar yang bisa digeser, *form* pencarian yang responsif, atau tombol "Load More".
3. **Dokumentasikan**: Buat **peta konsep** yang menggambarkan fungsionalitas dan teknologi yang Anda temukan. Inspeksi kode halaman web yang Anda amati, buat korelasi antara tampilan dan kode yang bersesuaian.

Petunjuk Penyelesaian

1. Buka browser dan kunjungi website pilihan Anda.
2. Tekan F12 atau klik kanan, lalu pilih **Inspect** untuk membuka **Developer Tools**.
3. Gunakan tab **Elements** untuk melihat struktur HTML dan **Styles** untuk melihat properti CSS yang diterapkan.
4. Perhatikan bagaimana setiap elemen diatur dan bagaimana interaksi bekerja. Contoh: Saat mengklik tombol "Like", perhatikan apakah ada perubahan di halaman.
5. Buat laporan pengamatan sesuai dengan worksheet yang tersedia pada tautan.

Hasil Akhir yang Diharapkan

Peta konsep yang jelas dan terstruktur yang menunjukkan pemahaman Anda tentang bagaimana ketiga teknologi dasar tersebut bekerja sama untuk membangun sebuah website yang fungsional. Anda dapat mempresentasikan hasil peta konsep ini pada pertemuan berikutnya untuk berdiskusi.

Worksheets:

1. Laporan pengamatan elemen-elemen website.

Pertemuan 2

Modul Pertemuan 2: HTML Dasar

Durasi: 1 Pertemuan

Tujuan Pembelajaran

Setelah menyelesaikan modul ini, mahasiswa diharapkan mampu:

- Memahami dan menjelaskan peran dasar HTML dalam struktur halaman web.
- Mengenali dan menggunakan tag-tag HTML dasar untuk membuat konten.
- Membangun struktur dasar dokumen HTML yang benar.
- Membuat halaman web statis sederhana dengan elemen-elemen HTML.

Materi: Struktur & Elemen Dasar HTML

HTML (**H**yper**T**ext **M**arkup **L**anguage) adalah bahasa *markup* yang digunakan untuk membangun struktur dasar dari sebuah halaman web. HTML terdiri dari serangkaian *tag* yang mendefinisikan elemen-elemen seperti judul, paragraf, gambar, dan tautan.

Struktur Dokumen HTML

Setiap dokumen HTML memiliki struktur dasar yang wajib diikuti.

- `<!DOCTYPE html>`: Deklarasi yang mendefinisikan versi HTML (saat ini HTML5).
- `<html>`: Elemen akar yang membungkus seluruh konten halaman web.
- `<head>`: Berisi metadata tentang dokumen (informasi yang tidak ditampilkan di halaman), seperti judul halaman yang muncul di tab browser (`<title>`) dan *link* ke CSS atau JavaScript.
- `<body>`: Berisi semua konten yang akan ditampilkan di halaman web, seperti teks, gambar, dan video.

Contoh struktur:

```
None

<!DOCTYPE html>
<html>
<head>
    <title>Judul Halaman Web Saya</title>
</head>
<body>
    <!-- Konten akan ditulis di sini -->
</body>
</html>
```

Elemen-elemen HTML Dasar

Berikut adalah beberapa tag HTML dasar yang paling sering digunakan:

- **Heading (`<h1>` - `<h6>`)**: Digunakan untuk judul atau sub-judul. `<h1>` memiliki ukuran terbesar, sementara `<h6>` adalah yang terkecil.
- **Paragraf (`<p>`)**: Digunakan untuk membuat paragraf teks.

- **Tautan (<a>)**: Digunakan untuk membuat *hyperlink*. Atribut href wajib digunakan untuk menentukan URL tujuan.
    - Contoh: Menuju Google
- **Gambar (<img>)**: Digunakan untuk menampilkan gambar. Atribut src wajib untuk menentukan lokasi file gambar, dan alt untuk teks alternatif jika gambar tidak bisa dimuat.
    - Contoh: <img src="gambar_saya.jpg" alt="Foto Diri Saya">
- **Daftar (<ul>, <ol>, <li>)**:
    - : Daftar tak berurut (menggunakan poin-poin).
    - : Daftar berurut (menggunakan angka).
    - : Item daftar.
- **Elemen Semantik**: Tag-tag ini memberikan makna pada konten, membantu mesin pencari dan *screen reader* memahami struktur halaman. Contohnya termasuk <header>, <nav>, <main>, <article>, dan <footer>.

Studi Kasus: Membuat Halaman Web Pribadi

Pada pertemuan ini, Anda akan menerapkan pengetahuan HTML dasar untuk membuat halaman web pribadi sederhana. Halaman ini akan berisi informasi tentang diri Anda, hobi, dan tautan media sosial.

Petunjuk Praktikum

1. Buka *text editor* Anda (misalnya VS Code).
2. Buat file baru dan simpan dengan nama index.html.
3. Tuliskan struktur dasar dokumen HTML seperti yang sudah dijelaskan di atas.
4. Di dalam tag <body>, buatlah:
    - Sebuah judul utama menggunakan tag <h1> berisi nama Anda.
    - Sebuah sub-judul menggunakan tag <h2> berisi profesi atau deskripsi singkat Anda.
    - Satu atau dua paragraf menggunakan tag <p> yang menceritakan tentang diri Anda.
    - Daftar hobi atau keahlian menggunakan tag <ul> atau <ol>.
    - Tambahkan foto profil Anda menggunakan tag <img>. Pastikan Anda memiliki URL atau file gambar yang benar.
    - Tambahkan tautan ke salah satu profil media sosial Anda menggunakan tag <a>.
5. Setelah selesai, simpan file index.html.
6. Buka file tersebut di browser Anda untuk melihat hasilnya.

Hasil Akhir yang Diharapkan

Sebuah halaman web statis yang menampilkan informasi pribadi Anda, terstruktur dengan baik menggunakan tag-tag HTML yang sudah dipelajari.

Petunjuk Pengumpulan Tugas

Untuk mengumpulkan tugas, ikuti langkah-langkah berikut:

1. Buat repositori pribadi di GitLab dengan nama pertemuan-2-html-dasar.
2. *Push* (unggah) seluruh file tugas Anda (index.html dan gambar jika ada) ke repositori tersebut.
3. Pastikan repositori Anda **bersifat privat** dan hanya bisa diakses oleh Anda.
4. Berikan akses Reporter kepada dosen Anda di repositori tersebut.
5. *Link* repositori GitLab tersebut akan dimasukkan ke dalam *worksheet* Anda.

Pertemuan 3

Modul Pertemuan 3: CSS Dasar

Tujuan Pembelajaran

Setelah menyelesaikan modul ini, mahasiswa diharapkan mampu:

- Memahami peran dan fungsi CSS dalam pengembangan web.
- Menerapkan CSS ke dalam dokumen HTML dengan tiga cara berbeda.
- Menggunakan *selector* CSS dasar untuk menargetkan elemen HTML.
- Mengubah tampilan visual elemen HTML dengan properti CSS dasar.
- Memahami dan menerapkan konsep *CSS Box Model*.
- Mengenali konsep dasar *layout*, *styling*, dan *typography* dalam CSS.
- Mengenal teknologi modern yang membantu penulisan CSS.

Materi: Mempercantik Tampilan dengan CSS

1. Pengertian dan Peran CSS

CSS (Cascading Style Sheets) adalah bahasa *stylesheet* yang digunakan untuk mendeskripsikan presentasi dari sebuah dokumen yang ditulis dalam bahasa *markup* seperti HTML. Dengan kata lain, jika HTML adalah kerangka bangunan, maka CSS adalah cat, furnitur, dan dekorasi yang membuatnya indah dan nyaman.

2. Cara Menambahkan CSS ke HTML

Ada tiga metode utama untuk menambahkan gaya CSS ke dokumen HTML:

1. **Inline CSS**: Ditulis langsung di dalam tag HTML menggunakan atribut `style`. Cara ini cocok untuk perubahan gaya yang sangat spesifik dan cepat, tetapi tidak disarankan untuk skala besar.

None

2.

```
<h1 style="color: blue; font-size: 24px;">Halo Dunia!</h1>
```

3.

4. **Internal CSS**: Ditulis di dalam tag `<style>` di bagian `<head>` dari dokumen HTML. Metode ini baik untuk gaya yang spesifik untuk satu halaman web.

None

5.

```
<head>
    <style>
        h1 {
            color: blue;
            font-size: 24px;
        }
    </style>
```

```
</head>
```

6.

7. **External CSS**: Ditulis dalam file terpisah dengan ekstensi `.css` dan dihubungkan ke dokumen HTML menggunakan tag `<link>`. Ini adalah metode yang paling direkomendasikan karena membuat kode lebih rapi dan mudah dikelola.

None

8.

```
<head>
    <link rel="stylesheet" href="style.css">
</head>
```

9.

3. Konsep Dasar CSS

- **Box Model**: Setiap elemen HTML dapat dianggap sebagai sebuah kotak.
    - **Content**: Konten aktual dari elemen (teks, gambar, dll.).
    - **Padding**: Ruang antara konten dan batas (*border*).
    - **Border**: Garis pembatas di sekitar *padding* dan konten.
    - Margin: Ruang di luar border, yang memisahkan elemen dari elemen lain. Memahami Box Model sangat penting untuk mengatur tata letak dan jarak antar elemen.
- **Layout**: Mengatur posisi dan tata letak elemen di halaman. Konsep dasar yang penting adalah **display** (`block`, `inline`, `inline-block`), **position** (`static`, `relative`, `absolute`, `fixed`), dan **float**. Untuk *layout* modern, dua teknik utama yang akan kita pelajari pada pertemuan berikutnya adalah **Flexbox** dan **CSS Grid**.
- **Typography**: Memastikan teks di halaman web mudah dibaca dan menarik secara visual. Properti penting meliputi `font-family`, `font-size`, `font-weight`, `line-height`, dan `text-align`.
- **Styling**: Mengacu pada properti visual seperti `color`, `background-color`, `border`, `box-shadow`, dan `border-radius`. Ini adalah properti yang paling sering Anda gunakan untuk membuat halaman web terlihat lebih menarik.

4. Teknologi Modern dalam CSS

Selain CSS murni, ada banyak teknologi yang diciptakan untuk mempermudah penulisan dan pengelolaan CSS, seperti **Sass**, **Less**, atau **Stylus**. Teknologi ini disebut **CSS *preprocessor***. Mereka menambahkan fitur-fitur seperti variabel, fungsi, dan *mixin* ke dalam CSS, yang kemudian akan dikompilasi menjadi CSS standar yang dapat dibaca oleh browser. Pada mata kuliah ini, kita akan fokus pada CSS dasar, namun penting untuk mengetahui bahwa teknologi ini ada.

Latihan

Kode Program untuk Latihan

HTML

```
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Latihan CSS</title>
    <style>
        * {
            box-sizing: border-box;
            margin: 0;
            padding: 0;
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
        }

        body {
            background-color: #f5f7fa;
            color: #333;
            line-height: 1.6;
            padding: 20px;
            max-width: 1200px;
            margin: 0 auto;
        }

        header {
            text-align: center;
            margin-bottom: 40px;
            padding: 20px;
            background: linear-gradient(135deg, #4a6fa5, #2c3e50);
            color: white;
            border-radius: 10px;
            box-shadow: 0 4px 6px rgba(0,0,0,0.1);
        }

        h1, h2, h3 {
            margin-bottom: 15px;
            color: #2c3e50;
        }

        .difficulty-section {
            margin-bottom: 40px;
            padding: 25px;
            background-color: white;
            border-radius: 10px;
            box-shadow: 0 2px 10px rgba(0,0,0,0.08);
        }
```

```css
.difficulty-title {
    border-bottom: 3px solid;
    padding-bottom: 10px;
    margin-bottom: 20px;
    display: flex;
    justify-content: space-between;
    align-items: center;
}

.easy .difficulty-title {
    border-color: #2ecc71;
    color: #27ae60;
}

.medium .difficulty-title {
    border-color: #f39c12;
    color: #e67e22;
}

.hard .difficulty-title {
    border-color: #e74c3c;
    color: #c0392b;
}

.exercise {
    margin-bottom: 30px;
    padding: 20px;
    border-radius: 8px;
    background-color: #f8f9fa;
    border-left: 4px solid #4a6fa5;
}

.instructions {
    background-color: #e8f4fc;
    padding: 15px;
    border-radius: 5px;
    margin-bottom: 20px;
}

.instructions h3 {
    color: #2c3e50;
    margin-bottom: 10px;
}

.workspace {
    border: 2px dashed #95a5a6;
    padding: 20px;
    min-height: 200px;
    margin-bottom: 15px;
    background-color: #fff;
```

```
}

.hint {
    background-color: #fdebd0;
    padding: 10px 15px;
    border-radius: 5px;
    font-size: 0.9em;
    border-left: 4px solid #f39c12;
    margin-top: 10px;
}

.solution-btn {
    background-color: #4a6fa5;
    color: white;
    border: none;
    padding: 8px 15px;
    border-radius: 5px;
    cursor: pointer;
    font-size: 0.9em;
    transition: background-color 0.3s;
}

.solution-btn:hover {
    background-color: #2c3e50;
}

.solution {
    display: none;
    margin-top: 15px;
    padding: 15px;
    background-color: #f9f9f9;
    border: 1px solid #ddd;
    border-radius: 5px;
}

.code {
    font-family: 'Courier New', monospace;
    background-color: #2c3e50;
    color: #ecf0f1;
    padding: 15px;
    border-radius: 5px;
    overflow-x: auto;
    margin: 10px 0;
    font-size: 0.9em;
}

footer {
    text-align: center;
    margin-top: 40px;
    padding: 20px;
```

```
            background-color: #2c3e50;
            color: white;
            border-radius: 10px;
        }

        @media (max-width: 768px) {
            body {
                padding: 10px;
            }

            .difficulty-section {
                padding: 15px;
            }
        }
    </style>
</head>
<body>
    <header>
        <h1>Latihan CSS</h1>
        <p>Tingkat Mudah hingga Kompleks</p>
    </header>

    <section class="difficulty-section easy">
        <div class="difficulty-title">
            <h2>Tingkat Mudah</h2>
            <span>⭐</span>
        </div>

        <div class="exercise">
            <div class="instructions">
                <h3>Latihan 1: Styling Teks</h3>
                <p>Buatlah paragraf dengan ketentuan:</p>
                <ul>
                    <li>Warna teks: #2c3e50</li>
                    <li>Ukuran font: 18px</li>
                    <li>Tebal font: 500</li>
                    <li>Jarak antar baris: 1.6</li>
                    <li>Rata kiri-kanan (justify)</li>
                </ul>
            </div>
            <div class="workspace">
                <!-- Kerjakan di sini -->
                <p>Ini adalah teks contoh. Ubah style teks ini sesuai dengan
instruksi di atas.</p>
            </div>
            <button class="solution-btn"
onclick="toggleSolution('solution1')">Tampilkan Solusi</button>
            <div id="solution1" class="solution">
                <p>CSS yang digunakan:</p>
                <div class="code">
```

```
                    p {<br>
                      color: #2c3e50;<br>
                      font-size: 18px;<br>
                      font-weight: 500;<br>
                      line-height: 1.6;<br>
                      text-align: justify;<br>
                    }
                </div>
            </div>
        </div>

        <div class="exercise">
            <div class="instructions">
                <h3>Latihan 2: Box Model</h3>
                <p>Buatlah sebuah div dengan ketentuan:</p>
                <ul>
                    <li>Lebar: 250px, Tinggi: 150px</li>
                    <li>Padding: 20px</li>
                    <li>Border: 3px solid #3498db</li>
                    <li>Margin: 15px</li>
                    <li>Background color: #ecf0f1</li>
                    <li>Border radius: 8px</li>
                </ul>
            </div>
            <div class="workspace">
                <!-- Kerjakan di sini -->
                <div>Ini adalah kotak. Style div ini sesuai dengan instruksi di
atas.</div>
            </div>
            <button class="solution-btn"
onclick="toggleSolution('solution2')">Tampilkan Solusi</button>
            <div id="solution2" class="solution">
                <p>CSS yang digunakan:</p>
                <div class="code">
                    div {<br>
                      width: 250px;<br>
                      height: 150px;<br>
                      padding: 20px;<br>
                      border: 3px solid #3498db;<br>
                      margin: 15px;<br>
                      background-color: #ecf0f1;<br>
                      border-radius: 8px;<br>
                    }
                </div>
            </div>
        </div>
    </section>

    <section class="difficulty-section medium">
        <div class="difficulty-title">
```

```
            <h2>Tingkat Menengah</h2>
            <span>⭐⭐</span>
        </div>

        <div class="exercise">
            <div class="instructions">
                <h3>Latihan 3: Flexbox Layout</h3>
                <p>Buatlah container flexbox dengan 3 kotak di dalamnya dengan
ketentuan:</p>
                <ul>
                    <li>Container: display flex, justify-content: space-between,
align-items: center</li>
                    <li>Setiap kotak: Lebar 80px, Tinggi 80px, Background #3498db,
Margin 5px</li>
                    <li>Kotak tengah: align-self: stretch (tinggi 120px)</li>
                </ul>
            </div>
            <div class="workspace">
                <!-- Kerjakan di sini -->
                <div class="flex-container">
                    <div class="box">1</div>
                    <div class="box middle">2</div>
                    <div class="box">3</div>
                </div>
            </div>
            <button class="solution-btn"
onclick="toggleSolution('solution3')">Tampilkan Solusi</button>
            <div id="solution3" class="solution">
                <p>CSS yang digunakan:</p>
                <div class="code">
                    .flex-container {<br>
                      display: flex;<br>
                      justify-content: space-between;<br>
                      align-items: center;<br>
                      height: 150px;<br>
                      border: 2px dashed #ccc;<br>
                      padding: 10px;<br>
                    }<br><br>

                    .box {<br>
                      width: 80px;<br>
                      height: 80px;<br>
                      background-color: #3498db;<br>
                      margin: 5px;<br>
                      display: flex;<br>
                      justify-content: center;<br>
                      align-items: center;<br>
                      color: white;<br>
                      font-weight: bold;<br>
                    }<br><br>
```

```
                    .middle {<br>
                      align-self: stretch;<br>
                      height: 120px;<br>
                    }
                </div>
            </div>
        </div>

        <div class="exercise">
            <div class="instructions">
                <h3>Latihan 4: Hover Effects</h3>
                <p>Buatlah sebuah tombol dengan efek hover dengan ketentuan:</p>
                <ul>
                    <li>Tombol: Padding 12px 24px, Background #2ecc71, Warna teks
putih, Border none, Border radius 4px, Font size 16px</li>
                    <li>Efek hover: Background #27ae60, Transform scale(1.05),
Transition 0.3s</li>
                    <li>Efek aktif: Transform scale(0.95)</li>
                </ul>
            </div>
            <div class="workspace">
                <!-- Kerjakan di sini -->
                <button class="custom-btn">Tombol Saya</button>
            </div>
            <button class="solution-btn"
onclick="toggleSolution('solution4')">Tampilkan Solusi</button>
            <div id="solution4" class="solution">
                <p>CSS yang digunakan:</p>
                <div class="code">
                    .custom-btn {<br>
                      padding: 12px 24px;<br>
                      background-color: #2ecc71;<br>
                      color: white;<br>
                      border: none;<br>
                      border-radius: 4px;<br>
                      font-size: 16px;<br>
                      cursor: pointer;<br>
                      transition: all 0.3s ease;<br>
                    }<br><br>

                    .custom-btn:hover {<br>
                      background-color: #27ae60;<br>
                      transform: scale(1.05);<br>
                    }<br><br>

                    .custom-btn:active {<br>
                      transform: scale(0.95);<br>
                    }
                </div>
```

```
            </div>
        </div>
    </section>

    <section class="difficulty-section hard">
        <div class="difficulty-title">
            <h2>Tingkat Kompleks</h2>
            <span>⭐⭐⭐</span>
        </div>

        <div class="exercise">
            <div class="instructions">
                <h3>Latihan 5: Card Component</h3>
                <p>Buatlah komponen kartu (card) dengan ketentuan:</p>
                <ul>
                    <li>Kartu: Lebar 300px, Border radius 10px, Overflow hidden,
Box shadow, Background putih</li>
                    <li>Gambar: Lebar 100%, Tinggi 180px, Object-fit cover</li>
                    <li>Konten: Padding 20px</li>
                    <li>Judul: Font size 20px, Margin bottom 10px</li>
                    <li>Tombol: Background #3498db, Warna putih, Padding 8px 16px,
Border radius 4px, Border none, Display inline-block</li>
                    <li>Efek hover pada tombol: Background #2980b9</li>
                </ul>
            </div>
            <div class="workspace">
                <!-- Kerjakan di sini -->
                <div class="card">
                    <img src="https://via.placeholder.com/300x180"
alt="Placeholder">
                    <div class="card-content">
                        <h3>Judul Kartu</h3>
                        <p>Ini adalah deskripsi kartu. Ini adalah contoh pembuatan
komponen kartu dengan HTML dan CSS.</p>
                        <button>Baca Selengkapnya</button>
                    </div>
                </div>
            </div>
            <button class="solution-btn"
onclick="toggleSolution('solution5')">Tampilkan Solusi</button>
            <div id="solution5" class="solution">
                <p>CSS yang digunakan:</p>
                <div class="code">
                    .card {<br>
                      width: 300px;<br>
                      border-radius: 10px;<br>
                      overflow: hidden;<br>
                      box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);<br>
                      background-color: white;<br>
                    }<br><br>
```

```
                .card img {<br>
                  width: 100%;<br>
                  height: 180px;<br>
                  object-fit: cover;<br>
                }<br><br>

                .card-content {<br>
                  padding: 20px;<br>
                }<br><br>

                .card-content h3 {<br>
                  font-size: 20px;<br>
                  margin-bottom: 10px;<br>
                  color: #2c3e50;<br>
                }<br><br>

                .card-content p {<br>
                  color: #7f8c8d;<br>
                  margin-bottom: 15px;<br>
                }<br><br>

                .card-content button {<br>
                  background-color: #3498db;<br>
                  color: white;<br>
                  padding: 8px 16px;<br>
                  border-radius: 4px;<br>
                  border: none;<br>
                  cursor: pointer;<br>
                  transition: background-color 0.3s;<br>
                }<br><br>

                .card-content button:hover {<br>
                  background-color: #2980b9;<br>
                }
            </div>
        </div>
    </div>

    <div class="exercise">
        <div class="instructions">
            <h3>Latihan 6: Responsive Navigation</h3>
            <p>Buatlah navigasi responsif dengan ketentuan:</p>
            <ul>
                <li>Navbar: Background #2c3e50, Padding 15px, Display flex,
Justify-content space-between, Align-items center</li>
                <li>Logo: Warna putih, Font size 20px, Font weight bold</li>
                <li>Menu: Display flex, List-style none</li>
                <li>Item menu: Margin 0 15px</li>
```

```
                    <li>Link: Warna putih, Text-decoration none, Transition
0.3s</li>
                    <li>Efek hover pada link: Warna #3498db</li>
                    <li>Tombol toggle: Display none secara default (untuk
mobile)</li>
                </ul>
            </div>
            <div class="workspace">
                <!-- Kerjakan di sini -->
                <nav class="navbar">
                    <div class="logo">Logo</div>
                    <ul class="nav-menu">
                        <li><a href="#">Beranda</a></li>
                        <li><a href="#">Tentang</a></li>
                        <li><a href="#">Layanan</a></li>
                        <li><a href="#">Kontak</a></li>
                    </ul>
                    <div class="toggle-btn">☰</div>
                </nav>
            </div>
            <button class="solution-btn"
onclick="toggleSolution('solution6')">Tampilkan Solusi</button>
            <div id="solution6" class="solution">
                <p>CSS yang digunakan:</p>
                <div class="code">
                    .navbar {<br>
                      background-color: #2c3e50;<br>
                      padding: 15px;<br>
                      display: flex;<br>
                      justify-content: space-between;<br>
                      align-items: center;<br>
                    }<br><br>

                    .logo {<br>
                      color: white;<br>
                      font-size: 20px;<br>
                      font-weight: bold;<br>
                    }<br><br>

                    .nav-menu {<br>
                      display: flex;<br>
                      list-style: none;<br>
                    }<br><br>

                    .nav-menu li {<br>
                      margin: 0 15px;<br>
                    }<br><br>

                    .nav-menu a {<br>
                      color: white;<br>
```

```
                  text-decoration: none;<br>
                  transition: color 0.3s;<br>
                }<br><br>

                .nav-menu a:hover {<br>
                  color: #3498db;<br>
                }<br><br>

                .toggle-btn {<br>
                  display: none;<br>
                  color: white;<br>
                  font-size: 24px;<br>
                  cursor: pointer;<br>
                }<br><br>

                /* Responsive styles */<br>
                @media (max-width: 768px) {<br>
                  .nav-menu {<br>
                    display: none;<br>
                    flex-direction: column;<br>
                    width: 100%;<br>
                    position: absolute;<br>
                    top: 60px;<br>
                    left: 0;<br>
                    background-color: #2c3e50;<br>
                    padding: 20px;<br>
                  }<br><br>

                  .nav-menu.active {<br>
                    display: flex;<br>
                  }<br><br>

                  .nav-menu li {<br>
                    margin: 10px 0;<br>
                  }<br><br>

                  .toggle-btn {<br>
                    display: block;<br>
                  }<br>
                }
            </div>
        </div>
    </div>
</section>

<footer>
    <p>© 2023 Latihan CSS - Dari Tingkat Mudah hingga Kompleks</p>
</footer>

<script>
```

```
        function toggleSolution(id) {
            const solution = document.getElementById(id);
            if (solution.style.display === 'block') {
                solution.style.display = 'none';
            } else {
                solution.style.display = 'block';
            }
        }
    </script>
</body>
</html>
```

Cara Menggunakan Latihan

1. Salin kode di atas dan simpan dalam file HTML
2. Buka file di browser web
3. Setiap latihan memiliki area workspace untuk menerapkan CSS
4. Kerjakan latihan sesuai dengan instruksi yang diberikan
5.
6. Gunakan tombol "Tampilkan Solusi" untuk melihat contoh solusi jika diperlukan

Studi Kasus: Menghias Halaman Web Pribadi

Tugas

Lanjutkan tugas dari pertemuan sebelumnya. Gunakan CSS untuk mendesain halaman web pribadi yang telah Anda buat dengan HTML. Terapkan gaya pada elemen-elemen berikut:

1. Gunakan warna latar belakang yang berbeda untuk bagian *header*, konten utama, dan *footer*.
2. Ubah jenis *font* dan ukuran teks.
3. Berikan efek visual pada tautan (misalnya, ganti warna saat kursor di atasnya).
4. Atur jarak antar elemen menggunakan `padding` dan `margin` untuk tampilan yang rapi.

Petunjuk Pengumpulan

1. Buat folder baru di repositori GitLab Anda dengan nama `pertemuan-3`.
2. Salin file `index.html` dari tugas Pertemuan 2 ke folder ini.
3. Buat file CSS eksternal baru bernama `style.css` di dalam folder `pertemuan-3`.
4. Hubungkan file `style.css` ke `index.html` menggunakan tag `<link>`.
5. Tulis kode CSS Anda di file `style.css` untuk menyelesaikan tugas.
6. Perbarui file `index.html` utama di *root* repositori Anda dengan menambahkan tautan ke tugas ini.
7. *Commit* dan *push* perubahan ke GitLab.

Pertemuan 4

Modul Pertemuan 4: CSS Lanjutan

Tujuan Pembelajaran

Setelah menyelesaikan modul ini, mahasiswa diharapkan mampu:

- Memahami perbedaan antara *layout* tradisional dan modern (Flexbox, Grid).
- Menerapkan Flexbox untuk membuat tata letak satu dimensi yang efisien.
- Menerapkan CSS Grid untuk membuat tata letak dua dimensi yang kompleks.
- Menggabungkan Flexbox dan CSS Grid untuk menciptakan desain web responsif.
- Menerapkan media queries untuk memastikan tata letak website beradaptasi dengan berbagai ukuran layar.

Materi: Merancang Tata Letak Fleksibel dan Responsif

1. Perkenalan Flexbox

Flexbox (atau *Flexible Box Layout*) adalah metode *layout* satu dimensi yang dirancang untuk mendistribusikan ruang dan menyelaraskan item dalam satu baris atau satu kolom. Flexbox sangat cocok untuk membuat komponen seperti menu navigasi, galeri foto, atau tata letak form yang rapi.

- **Konsep Utama**: Flexbox bekerja dengan dua jenis elemen:
    - **Flex Container**: Elemen induk yang menampung *flex item*. Properti Flexbox diterapkan pada elemen ini.
    - **Flex Item**: Elemen anak di dalam Flex Container.
- **Properti Penting**:
    - `display: flex;`: Properti ini mengubah sebuah elemen menjadi Flex Container.
    - `justify-content`: Mengatur perataan horizontal item.
    - `align-items`: Mengatur perataan vertikal item.
    - `flex-direction`: Mengatur arah item (baris atau kolom).

2. Perkenalan CSS Grid

CSS Grid adalah sistem *layout* dua dimensi yang memungkinkan Anda untuk mengatur item dalam baris dan kolom secara bersamaan. Grid ideal untuk tata letak halaman web secara keseluruhan.

- **Konsep Utama**:
    - **Grid Container**: Elemen induk yang menampung *grid item*. Properti Grid diterapkan di sini.
    - **Grid Item**: Elemen anak di dalam Grid Container.
    - **Grid Lines, Tracks, Cells, & Areas**: Komponen yang membentuk struktur Grid.
- **Properti Penting**:
    - `display: grid;`: Mengubah elemen menjadi Grid Container.
    - `grid-template-columns`: Menentukan jumlah dan lebar kolom.
    - `grid-template-rows`: Menentukan jumlah dan tinggi baris.
    - `grid-gap`: Memberikan jarak antar sel Grid.

3. Membuat Tata Letak Responsif dengan Media Queries

Media Queries adalah fitur CSS yang memungkinkan Anda menerapkan gaya berbeda berdasarkan karakteristik perangkat, seperti lebar layar. Ini adalah fondasi dari desain web responsif.

- **Sintaks Dasar**:

None

-

```
@media (max-width: 600px) {
  .container {
    flex-direction: column;
  }
}
```

-

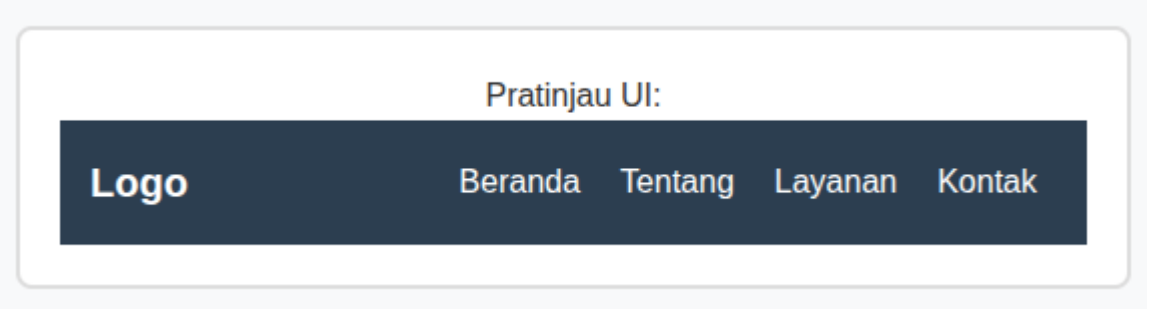
- Kode di atas akan mengubah arah *flex item* menjadi kolom ketika lebar layar kurang dari 600px, sangat berguna untuk tampilan seluler.

Latihan

Latihan 1: Navigasi Bar Horizontal

Buatlah navigasi bar horizontal dengan logo di kiri dan menu di kanan.
Kriteria:

- Warna latar: #2c3e50
- Teks warna putih
- Logo di kiri, menu di kanan
- Menu terdiri dari Beranda, Tentang, Layanan, Kontak



HTML

```
<div class="navbar">
                    <div class="logo">Logo</div>
                    <ul class="menu">
                        <li><a href="#">Beranda</a></li>
                        <li><a href="#">Tentang</a></li>
                        <li><a href="#">Layanan</a></li>
                        <li><a href="#">Kontak</a></li>
```

```
                    </ul>
 </div>
```

CSS

```css
.navbar {
    background-color: #2c3e50;
    padding: 15px;
    display: flex;
    justify-content: space-between;
    align-items: center;
}

.navbar .logo {
    color: white;
    font-weight: bold;
    font-size: 20px;
}

.navbar ul {
    display: flex;
    list-style: none;
}

.navbar li {
    margin: 0 10px;
}

.navbar a {
    color: white;
    text-decoration: none;
}
```

Latihan 2: Card Produk

Buatlah card produk dengan gambar, judul, deskripsi, dan tombol.
Kriteria:

- Lebar card: 300px
- Border radius: 8px
- Box shadow
- Terdapat gambar, judul, deskripsi, dan tombol

Pratinjau UI:



Nama Produk

Deskripsi singkat produk ini. Cocok untuk berbagai keperluan.

Beli Sekarang

HTML

```
<div class="card">
                    <img src="https://picsum.photos/300/300" alt="Gambar Produk">
                    <div class="content">
                        <h3>Nama Produk</h3>
                        <p>Deskripsi singkat produk ini. Cocok untuk berbagai
keperluan.</p>
                        <button>Beli Sekarang</button>
                    </div>
                </div>
```

CSS

```
.card {
    width: 300px;
    background-color: white;
    border-radius: 8px;
    overflow: hidden;
    box-shadow: 0 2px 10px rgba(0,0,0,0.1);
}

.card img {
```

```
    width: 100%;
    height: 160px;
    object-fit: cover;
}

.content {
    padding: 15px;
}

.content h3 {
    margin-bottom: 10px;
    color: #2c3e50;
}

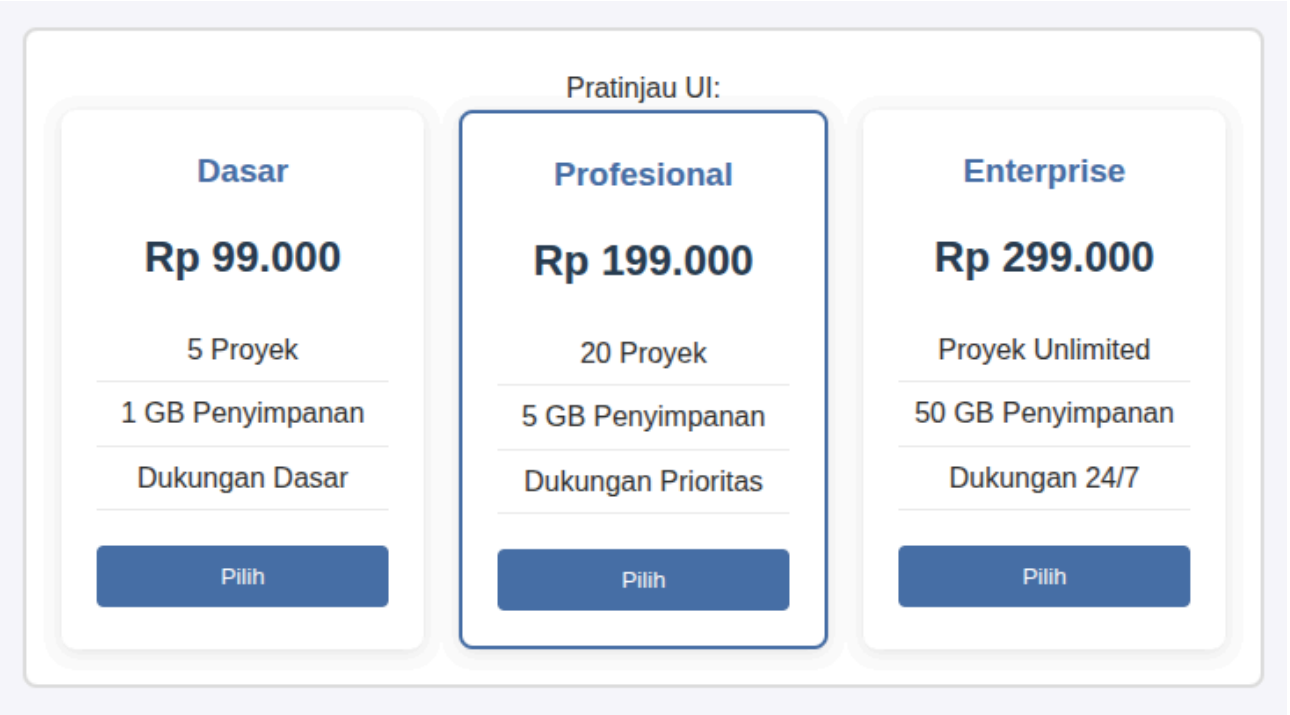
.content p {
    color: #666;
    margin-bottom: 15px;
    line-height: 1.4;
}

.content button {
    background-color: #4a6fa5;
    color: white;
    border: none;
    padding: 8px 15px;
    border-radius: 4px;
    cursor: pointer;
}
```

Latihan 3: Pricing Cards

Buatlah tiga pricing cards yang disejajarkan secara horizontal.
Kriteria:

- Menggunakan flexbox untuk menyusun cards
- Setiap card memiliki header, harga, fitur, dan tombol
- Card tengah memiliki penekanan (highlight)
- Responsif pada layar kecil



HTML

```
<div class="pricing-container">
                    <div class="pricing-card">
                        <h3>Dasar</h3>
                        <div class="price">Rp 99.000</div>
                        <ul class="features">
                            <li>5 Proyek</li>
                            <li>1 GB Penyimpanan</li>
                            <li>Dukungan Dasar</li>
                        </ul>
                        <button>Pilih</button>
                    </div>
                    <div class="pricing-card" style="border: 2px solid #4a6fa5;">
                        <h3>Profesional</h3>
                        <div class="price">Rp 199.000</div>
                        <ul class="features">
                            <li>20 Proyek</li>
                            <li>5 GB Penyimpanan</li>
                            <li>Dukungan Prioritas</li>
                        </ul>
                        <button>Pilih</button>
                    </div>
                    <div class="pricing-card">
                        <h3>Enterprise</h3>
                        <div class="price">Rp 299.000</div>
                        <ul class="features">
                            <li>Proyek Unlimited</li>
                            <li>50 GB Penyimpanan</li>
                            <li>Dukungan 24/7</li>
```

```
                    </ul>
                    <button>Pilih</button>
                </div>
            </div>
```

CSS

```css
.pricing-container {
    display: flex;
    justify-content: center;
    gap: 20px;
    padding: 20px;
}

.pricing-card {
    background-color: white;
    border-radius: 8px;
    padding: 20px;
    text-align: center;
    box-shadow: 0 2px 10px rgba(0,0,0,0.1);
    flex: 1;
    max-width: 250px;
}

.pricing-card.featured {
    border: 2px solid #4a6fa5;
    transform: scale(1.05);
}

.pricing-card h3 {
    color: #4a6fa5;
    margin-bottom: 15px;
}

.price {
    font-size: 24px;
    font-weight: bold;
    color: #2c3e50;
    margin-bottom: 15px;
}

.pricing-card ul {
    list-style: none;
    margin-bottom: 20px;
}

.pricing-card li {
    padding: 5px 0;
```

```
    border-bottom: 1px solid #eee;
}

.pricing-card button {
    background-color: #4a6fa5;
    color: white;
    border: none;
    padding: 10px 20px;
    border-radius: 4px;
    cursor: pointer;
    width: 100%;
}

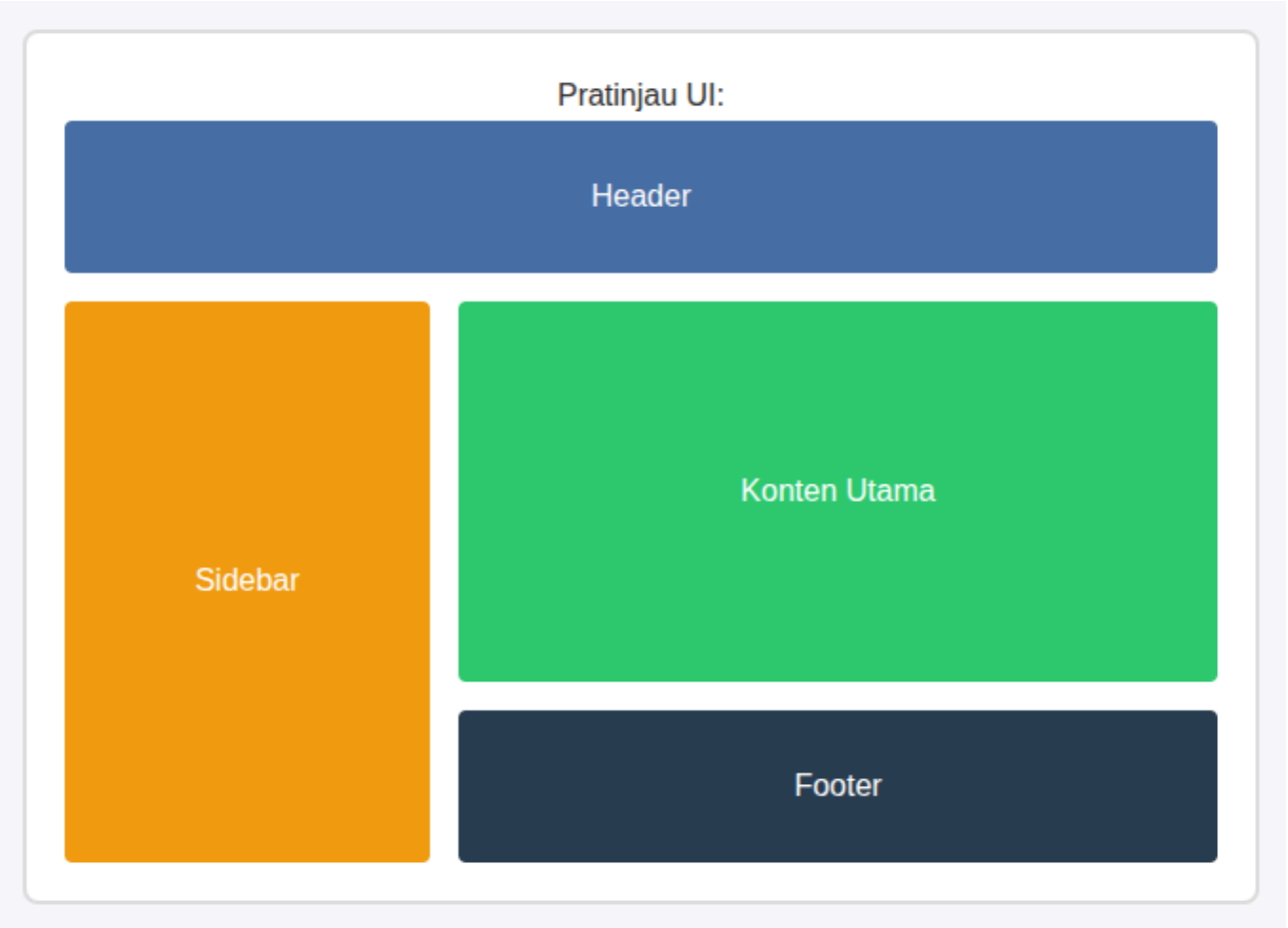
@media (max-width: 768px) {
    .pricing-container {
        flex-direction: column;
        align-items: center;
    }

    .pricing-card.featured {
        transform: none;
    }
}
```

Latihan 4: Layout dengan CSS Grid

Buatlah layout website sederhana menggunakan CSS Grid.
Kriteria:

- Header, sidebar, konten utama, dan footer
- Sidebar di kiri, konten di kanan
- Footer memanjang di bawah sidebar dan konten
- Gunakan CSS Grid untuk mengatur layout



HTML

```
<div class="grid-container">
                    <div class="header">Header</div>
                    <div class="sidebar">Sidebar</div>
                    <div class="main">Konten Utama</div>
                    <div class="footer">Footer</div>
                </div>
```

CSS

```
.grid-container {
    display: grid;
    grid-template-columns: 200px 1fr;
    grid-template-rows: 80px 1fr 80px;
    grid-template-areas:
        "header header"
        "sidebar main"
        "footer footer";
    gap: 15px;
    height: 400px;
}
```

```
header {
    grid-area: header;
    background-color: #4a6fa5;
    color: white;
    display: flex;
    align-items: center;
    justify-content: center;
    border-radius: 4px;
}

aside {
    grid-area: sidebar;
    background-color: #f39c12;
    color: white;
    display: flex;
    align-items: center;
    justify-content: center;
    border-radius: 4px;
}

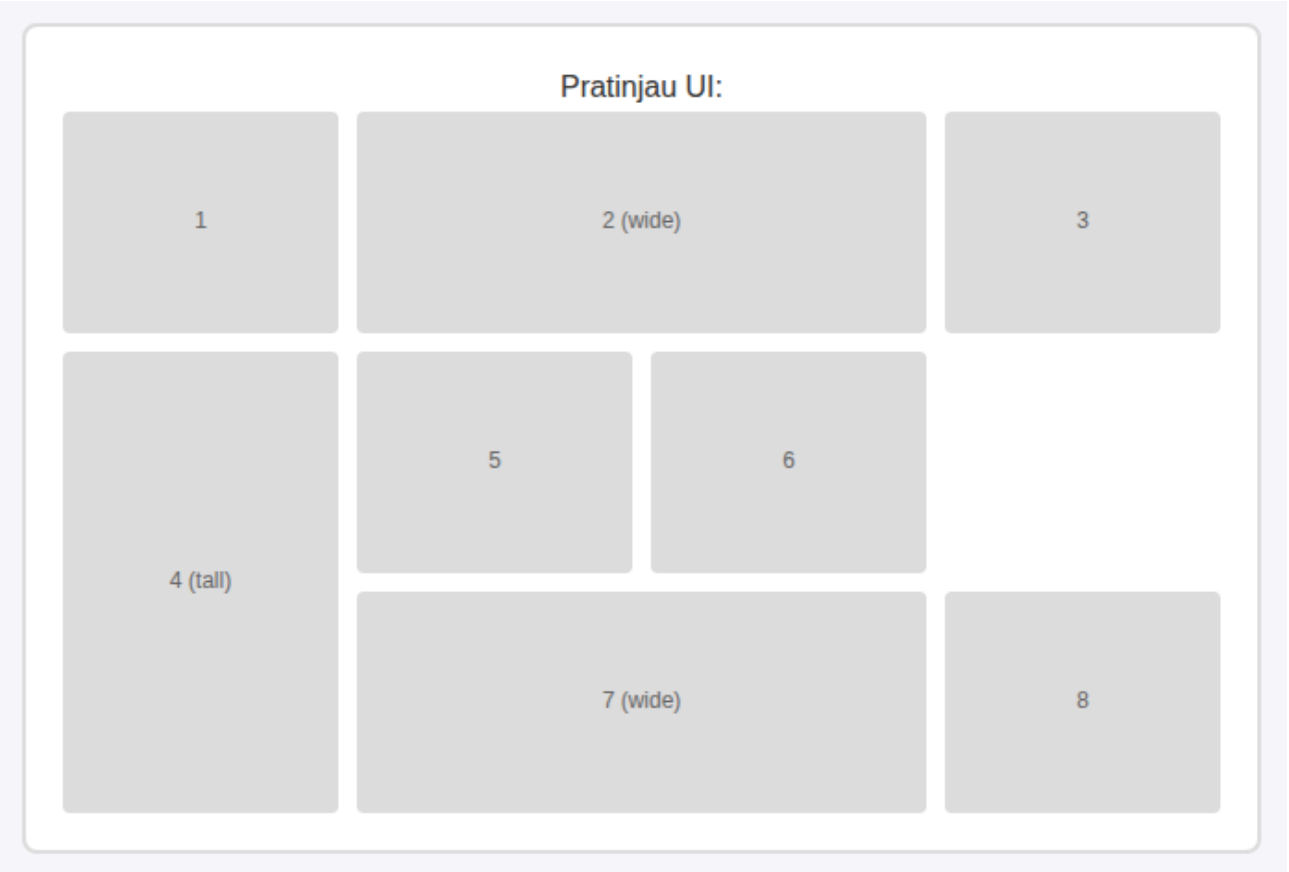
main {
    grid-area: main;
    background-color: #2ecc71;
    color: white;
    display: flex;
    align-items: center;
    justify-content: center;
    border-radius: 4px;
}

footer {
    grid-area: footer;
    background-color: #2c3e50;
    color: white;
    display: flex;
    align-items: center;
    justify-content: center;
    border-radius: 4px;
}
```

Latihan 5: Galeri Foto Responsif

Buatlah galeri foto responsif dengan CSS Grid.
Kriteria:

- Gunakan CSS Grid untuk membuat layout galeri
- Beberapa foto berukuran lebih besar (span multiple columns/rows)
- Responsif pada berbagai ukuran layar



HTML

```
<div class="gallery">
                    <div class="gallery-item">1</div>
                    <div class="gallery-item wide">2</div>
                    <div class="gallery-item">3</div>
                    <div class="gallery-item tall">4</div>
                    <div class="gallery-item">5</div>
                    <div class="gallery-item">6</div>
                    <div class="gallery-item wide">7</div>
                    <div class="gallery-item">8</div>
                </div>
```

CSS

```
.gallery {
    display: grid;
    grid-template-columns: repeat(4, 1fr);
    grid-template-rows: repeat(3, 120px);
    gap: 10px;
}

.gallery-item {
```

```
    background-color: #ddd;
    border-radius: 4px;
    display: flex;
    align-items: center;
    justify-content: center;
    color: #666;
    font-size: 12px;
}

.wide {
    grid-column: span 2;
}

.tall {
    grid-row: span 2;
}

@media (max-width: 768px) {
    .gallery {
        grid-template-columns: repeat(2, 1fr);
    }
}

@media (max-width: 480px) {
    .gallery {
        grid-template-columns: 1fr;
    }

    .wide, .tall {
        grid-column: span 1;
        grid-row: span 1;
    }
}
```

Studi Kasus: Membuat Halaman Web Responsif

Tugas

Lanjutkan tugas dari pertemuan sebelumnya. Ubah halaman web pribadi Anda menjadi desain yang responsif dengan menerapkan Flexbox dan CSS Grid.

1. **Layout Utama (Desktop)**: Gunakan **CSS Grid** untuk mengatur tata letak utama halaman Anda menjadi tiga bagian: *header*, konten utama, dan *footer*.
2. **Layout Komponen (Desktop)**: Di dalam bagian konten utama, gunakan **Flexbox** untuk mengatur tata letak elemen-elemen seperti kartu portofolio, galeri, atau daftar kontak agar tersusun rapi secara horizontal.
3. **Layout Responsif (Mobile)**: Gunakan **Media Queries** untuk mengubah tata letak. Saat diakses dari layar kecil (ponsel), pastikan semua elemen (misalnya, kartu portofolio) tersusun secara vertikal.

Petunjuk Pengumpulan

1. Buat folder baru di repositori GitLab Anda dengan nama `pertemuan-4`.
2. Salin file `index.html` dan `style.css` dari tugas Pertemuan 3 ke folder ini.
3. Perbarui file `style.css` untuk menerapkan Flexbox, CSS Grid, dan Media Queries sesuai dengan tugas.
4. Perbarui file `index.html` utama di *root* repositori dengan menambahkan tautan ke tugas ini.
5. *Commit* dan *push* perubahan ke GitLab.

Pertemuan 5

Modul Pertemuan 5: Framework UI

Tujuan Pembelajaran

Setelah menyelesaikan modul ini, mahasiswa diharapkan mampu:

- Memahami konsep dan manfaat dari *framework* CSS (UI).
- Memilih antara *framework* UI yang berbeda (contoh: Bootstrap vs. Tailwind CSS) sesuai kebutuhan proyek.
- Mengintegrasikan *framework* UI ke dalam proyek web.
- Menerapkan komponen dan kelas dari *framework* UI untuk membangun antarmuka pengguna yang responsif.

Materi: Mempercepat Pengembangan dengan Framework UI

Pengembangan web modern sering kali melibatkan penggunaan *framework* UI. Ini adalah kumpulan kelas CSS dan komponen JavaScript yang sudah jadi, dirancang untuk mempercepat proses pembuatan tampilan web yang konsisten dan responsif.

1. Apa itu Framework UI?

Framework UI adalah kerangka kerja yang menyediakan "shortcut" untuk menata dan mempercantik halaman web. Daripada menulis CSS dari nol untuk setiap elemen, Anda bisa menggunakan kelas yang sudah disediakan oleh *framework*.

2. Bootstrap vs. Tailwind CSS

Dua *framework* UI yang paling populer adalah Bootstrap dan Tailwind CSS. Keduanya memiliki filosofi yang berbeda:

- **Bootstrap**: Dikenal sebagai *component-first framework*. Bootstrap menyediakan komponen-komponen yang sudah jadi, seperti *cards*, tombol, dan navigasi. Anda hanya perlu menambahkan kelas seperti `btn` atau `card` untuk mendapatkan tampilan yang sudah terdesain.
  Kelebihan: Cepat untuk prototyping, tampilan konsisten, dan dokumentasi yang sangat lengkap. *Kekurangan*: Cenderung menghasilkan tampilan yang mirip di banyak website jika tidak disesuaikan.
- **Tailwind CSS**: Dikenal sebagai *utility-first framework*. Tailwind tidak menyediakan komponen jadi. Sebaliknya, ia menyediakan kelas-kelas utilitas yang sangat spesifik, seperti `flex`, `pt-4` (padding top 4), `text-center`, dan `bg-blue-500`. Anda membangun komponen Anda sendiri dengan menggabungkan kelas-kelas ini.
  Kelebihan: Kontrol penuh atas desain, ukuran file CSS akhir yang lebih kecil, dan fleksibilitas tak terbatas. *Kekurangan*: Membutuhkan waktu lebih lama untuk menulis kode jika dibandingkan dengan Bootstrap untuk tampilan yang sama.

3. Cara Mengintegrasikan Framework

Anda dapat menambahkan *framework* UI ke proyek Anda dengan dua cara umum:

1. **Menggunakan CDN (*Content Delivery Network*)**: Ini adalah cara termudah dan tercepat. Anda hanya perlu menyalin tautan CSS dan JavaScript dari website resmi *framework* dan

menaruhnya di dalam tag <head> file HTML Anda.
Contoh Pemasangan CDN:

Bootstrap:

```HTML
<head>
  <!-- Bootstrap CSS -->
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstr
ap.min.css" rel="stylesheet"
integrity="sha384-QWTKZyjpPEjISv5WaRU9OFeRpok6YctnYmDr5pNlyT2bRjXh0
JMhjY6hW+ALEwIH" crossorigin="anonymous">
</head>
<body>
  <!-- Konten HTML Anda di sini -->
  <!-- Bootstrap JS -->
  <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap
.bundle.min.js"
integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9GkcI
dslK1eN7N6jIeHz" crossorigin="anonymous"></script>
</body>
```

Tailwind CSS:

```CSS
<head>
  <!-- Tailwind CSS -->
  <script src="https://cdn.tailwindcss.com"></script>
</head>
```

```
<body>

  <!-- Konten HTML Anda di sini -->

</body>
```

2. **Instalasi Melalui NPM**: Ini adalah cara yang lebih canggih, sering digunakan dalam proyek-proyek besar yang menggunakan *tools* seperti Node.js atau React.

Untuk referensi lebih lanjut, Anda dapat mengunjungi dokumentasi resmi:

- **Dokumentasi Bootstrap**: https://getbootstrap.com/docs/5.3/getting-started/introduction/
- **Dokumentasi Tailwind CSS**: https://tailwindcss.com/docs/installation

Latihan

Latihan 1: Tombol dengan Tailwind

Buatlah tombol yang memiliki properti berikut menggunakan kelas Tailwind:

- Latar Belakang biru (`bg-blue-500`)
- Warna teks putih (`text-white`)
- Padding 8px vertikal dan 16px horizontal (`py-2 px-4`)
- Radius border 4px (`rounded`)
- Efek hover: latar belakang lebih gelap (`hover:bg-blue-700`)

```
HTML
<button id="button-exercise" class="">Klik Saya</button>
```

Latihan 2: Kartu Profil

Buatlah kartu profil dengan properti berikut:

- Border abu-abu dengan radius 8px
- Shadow pada hover
- Padding 24px
- Tata letak flex dengan konten terpusat secara vertikal
- Gambar dengan ukuran 64x64px dengan border bulat

Pratinjau:



Nama Pengguna
Deskripsi singkat pengguna

✓

HTML

```
<div id="card-exercise" class="border border-gray-300 rounded-lg p-6 flex
items-center hover:shadow-md">
    <img src="https://picsum.photos/200/200" alt="Profile" class="Tambahkan kelas
untuk gambar">
    <div class="ml-4">
        <h4 class="font-semibold">Nama Pengguna</h4>
        <p class="text-gray-600">Deskripsi singkat pengguna</p>
    </div>
</div>
```

Latihan 3: Grid Responsif

Buatlah grid responsive dengan 3 kolom pada layar besar, 2 kolom pada layar medium, dan 1 kolom pada layar kecil:

Pratinjau:

Item 1

Item 2

Item 3

✓

HTML

```
<div id="grid-exercise" class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3
gap-4">
    <div class="bg-blue-500 text-white p-4 rounded">Item 1</div>
```

```
    <div class="bg-blue-500 text-white p-4 rounded">Item 2</div>
    <div class="bg-blue-500 text-white p-4 rounded">Item 3</div>
</div>
```

Studi Kasus: Membuat Portofolio Sederhana

Tugas

Buatlah halaman portofolio sederhana yang memuat informasi diri Anda, keahlian, dan beberapa proyek yang pernah Anda kerjakan. Terapkan *framework* UI pilihan Anda (Bootstrap atau Tailwind CSS) untuk membuat tampilan yang menarik dan responsif.

Petunjuk Praktikum

1. Pilih *framework* UI yang ingin Anda gunakan.
2. Integrasikan *framework* tersebut ke dalam file `index.html` Anda menggunakan metode CDN.
3. Buat struktur HTML yang rapi untuk bagian-bagian berikut:
    - **Header**: Berisi nama dan judul Anda.
    - **About Me**: Berisi foto dan deskripsi singkat.
    - **Skills**: Berisi daftar keahlian Anda (misalnya, HTML, CSS, JavaScript).
    - **Projects**: Berisi beberapa "kartu" proyek dengan gambar, judul, dan deskripsi, ditampilkan dalam grid responsif.
    - **Footer**: Berisi tautan ke media sosial.
4. Gunakan kelas-kelas dari *framework* UI pilihan Anda untuk menata layout, memberikan warna, mengatur *font*, dan membuat tampilan menjadi responsif.

Petunjuk Pengumpulan

1. Buat folder baru di repository GitHub Anda dengan nama `pertemuan-7`.
2. Buat file `index.html` dan `style.css` (jika menggunakan Tailwind) di dalam folder ini.
3. Perbarui file `index.html` utama di *root* repository Anda dengan menambahkan tautan ke tugas ini.
4. *Commit* dan *push* perubahan ke GitLab.

Pertemuan 6

Modul Pertemuan 6: JavaScript Dasar

Tujuan Pembelajaran

Setelah menyelesaikan modul ini, mahasiswa diharapkan mampu:

- Memahami peran dan fungsi JavaScript sebagai bahasa pemrograman web.
- Menerapkan JavaScript ke dalam dokumen HTML dengan cara yang benar.
- Menulis kode JavaScript menggunakan sintaks dasar, termasuk variabel dan tipe data.
- Mampu menggunakan operator aritmatika dan logika.
- Membuat dan memanggil fungsi sederhana.
- Menggunakan *browser console* untuk *debugging* kode.

Materi: Menghidupkan Web dengan JavaScript

1. Pengenalan JavaScript

JavaScript adalah bahasa pemrograman yang digunakan untuk membuat halaman web menjadi interaktif. Jika HTML adalah kerangka dan CSS adalah tampilan, maka JavaScript adalah "otak" yang memungkinkan halaman untuk berinteraksi dengan pengguna, memanipulasi konten, dan berkomunikasi dengan server.

2. Cara Menambahkan JavaScript ke HTML

Ada dua cara utama untuk menyertakan kode JavaScript di halaman web:

Internal JavaScript: Ditulis di dalam tag `<script>` yang ditempatkan di dalam dokumen HTML, biasanya di bagian `<head>` atau di akhir `<body>`.

```
None
<script>
  // Kode JavaScript di sini
</script>
```

External JavaScript: Kode ditulis dalam file terpisah dengan ekstensi `.js` dan dihubungkan ke dokumen HTML menggunakan tag `<script>` dengan atribut `src`. Ini adalah metode yang disarankan untuk menjaga kode tetap rapi.

```
None
<script src="path/ke/file_anda.js"></script>
```

Praktik Terbaik: Selalu tempatkan tag `<script>` (untuk file eksternal) tepat sebelum tag penutup `</body>`. Ini memastikan semua elemen HTML sudah dimuat sebelum skrip mencoba memanipulasinya.

Sintaks Dasar JavaScript

- **Variabel**: Digunakan untuk menyimpan data. Anda bisa mendeklarasikannya dengan `let` atau `const`.
    - `let`: Variabel yang nilainya bisa diubah.
    - `const`: Variabel yang nilainya tidak bisa diubah (*constant*).

- **Tipe Data**: JavaScript memiliki beberapa tipe data dasar, seperti:
    - `String`: Teks (`"Halo, dunia!"`)
    - `Number`: Angka (`10`, `3.14`)
    - `Boolean`: Nilai benar atau salah (`true`, `false`)
- **Operator**: Simbol yang melakukan operasi pada nilai.
    - **Operator Aritmatika**: `+` (tambah), `-` (kurang), `*` (kali), `/` (bagi).
    - **Operator Perbandingan**: `==` (sama dengan), `!=` (tidak sama dengan), `>` (lebih besar dari), `<` (lebih kecil dari).
    - **Operator Logika**: `&&` (AND), `||` (OR), `!` (NOT).
- **Fungsi**: Blok kode yang dirancang untuk melakukan tugas tertentu. Fungsi akan berjalan saat dipanggil.

None

-

```
function tambah(a, b) {
  return a + b;
}

let hasil = tambah(5, 3);
console.log(hasil); // Output: 8
```

-

Debugging dengan Console

Anda dapat menggunakan **Browser Developer Tools** (tekan F12) untuk melihat pesan kesalahan atau mencatat nilai variabel menggunakan perintah `console.log()`. Ini adalah alat yang sangat penting untuk menemukan dan memperbaiki kesalahan dalam kode Anda.

Latihan

Latihan 1: Variabel dan Tipe Data

Buatlah variabel untuk menyimpan data berikut:

- Nama (string)
- Umur (number)
- Status (boolean - true jika sudah menikah, false jika belum)
- Tampilkan data tersebut di console

JavaScript

```
let nama = "John Doe";
let umur = 25;
let statusMenikah = false;

console.log("Nama: " + nama);
console.log("Umur: " + umur);
```

```
console.log("Status Menikah: " + statusMenikah);
```

Latihan 2: Operator dan Fungsi

Buatlah fungsi untuk menghitung luas persegi panjang.

- Fungsi harus menerima dua parameter: panjang dan lebar
- Gunakan operator matematika untuk menghitung luas
- Kembalikan hasil perhitungan
- Panggil fungsi dengan nilai yang berbeda dan tampilkan hasilnya

JavaScript

```
function hitungLuas(panjang, lebar) {
    return panjang * lebar;
}

let luas1 = hitungLuas(5, 8);
console.log("Luas persegi panjang dengan panjang 5 dan lebar 8 adalah: " + luas1);

let luas2 = hitungLuas(10, 3);
console.log("Luas persegi panjang dengan panjang 10 dan lebar 3 adalah: " +
luas2);
```

Latihan 3: Manipulasi DOM Dasar

Buatlah program yang memungkinkan pengguna mengubah tampilan halaman:

- Buat tombol untuk mengubah warna latar belakang
- Buat tombol untuk mengubah warna teks
- Buat tombol untuk menambah/mengurangi ukuran font
- Gunakan document.getElementById() atau document.querySelector()

JavaScript

```
// Membuat tombol
let btnBackground = document.createElement("button");
btnBackground.textContent = "Ubah Latar Belakang";
btnBackground.onclick = function() {
    document.body.style.backgroundColor = "#e8f4fc";
};

let btnTextColor = document.createElement("button");
btnTextColor.textContent = "Ubah Warna Teks";
btnTextColor.onclick = function() {
    document.body.style.color = "#2c3e50";
};
```

```
let btnIncreaseFont = document.createElement("button");
btnIncreaseFont.textContent = "Perbesar Font";
btnIncreaseFont.onclick = function() {
    let currentSize = parseInt(getComputedStyle(document.body).fontSize);
    document.body.style.fontSize = (currentSize + 1) + "px";
};

// Menambahkan tombol ke output
let outputDiv = document.getElementById("output3");
outputDiv.appendChild(btnBackground);
outputDiv.appendChild(btnTextColor);
outputDiv.appendChild(btnIncreaseFont);
```

Latihan 4: Event Handling

Buatlah form input dengan validasi:

- Buat input field untuk nama dan email
- Buat tombol submit
- Validasi bahwa nama tidak boleh kosong
- Validasi bahwa email harus mengandung karakter '@'
- Tampilkan pesan error jika validasi gagal

JavaScript

```
// Membuat form
let form = document.createElement("form");

// Input nama
let labelNama = document.createElement("label");
labelNama.textContent = "Nama: ";
let inputNama = document.createElement("input");
inputNama.type = "text";
inputNama.id = "nama";

// Input email
let labelEmail = document.createElement("label");
labelEmail.textContent = "Email: ";
let inputEmail = document.createElement("input");
inputEmail.type = "email";
inputEmail.id = "email";

// Tombol submit
let btnSubmit = document.createElement("button");
btnSubmit.type = "button";
btnSubmit.textContent = "Submit";

// Area pesan error
let errorDiv = document.createElement("div");
```

```
errorDiv.id = "error-message";
errorDiv.style.color = "red";

// Menambahkan event listener
btnSubmit.onclick = function() {
    let nama = inputNama.value;
    let email = inputEmail.value;
    let errorMessage = "";

    if (nama === "") {
        errorMessage += "Nama tidak boleh kosong. ";
    }

    if (email.indexOf("@") === -1) {
        errorMessage += "Email harus mengandung @. ";
    }

    if (errorMessage !== "") {
        errorDiv.textContent = errorMessage;
    } else {
        errorDiv.textContent = "Form valid! Data berhasil dikirim.";
        errorDiv.style.color = "green";
    }
};

// Menambahkan elemen ke form
form.appendChild(labelNama);
form.appendChild(inputNama);
form.appendChild(document.createElement("br"));
form.appendChild(labelEmail);
form.appendChild(inputEmail);
form.appendChild(document.createElement("br"));
form.appendChild(btnSubmit);
form.appendChild(errorDiv);

// Menambahkan form ke output
let outputDiv = document.getElementById("output4");
outputDiv.appendChild(form);
```

Latihan 5: Aplikasi To-Do List

Buatlah aplikasi to-do list sederhana dengan fitur berikut:

- Input untuk menambah tugas baru
- Tombol untuk menambahkan tugas ke daftar
- Daftar tugas yang dapat dicentang ketika selesai
- Tombol untuk menghapus tugas dari daftar
- Penyimpanan data di localStorage

JavaScript

```javascript
// Membuat elemen input dan tombol
let inputTask = document.createElement("input");
inputTask.type = "text";
inputTask.placeholder = "Masukkan tugas baru...";
inputTask.id = "task-input";

let btnAdd = document.createElement("button");
btnAdd.textContent = "Tambah";
btnAdd.id = "add-task";

let taskList = document.createElement("ul");
taskList.id = "task-list";

// Menambahkan event listener
btnAdd.onclick = function() {
    let taskText = inputTask.value.trim();
    if (taskText !== "") {
        addTask(taskText);
        inputTask.value = "";
        saveTasks();
    }
};

// Fungsi untuk menambahkan tugas
function addTask(taskText) {
    let li = document.createElement("li");

    let checkbox = document.createElement("input");
    checkbox.type = "checkbox";
    checkbox.addEventListener("change", function() {
        if (this.checked) {
            taskSpan.style.textDecoration = "line-through";
        } else {
            taskSpan.style.textDecoration = "none";
        }
        saveTasks();
    });

    let taskSpan = document.createElement("span");
    taskSpan.textContent = taskText;

    let deleteBtn = document.createElement("button");
    deleteBtn.textContent = "Hapus";
    deleteBtn.addEventListener("click", function() {
        li.remove();
        saveTasks();
    });

    li.appendChild(checkbox);
    li.appendChild(taskSpan);
```

```
    li.appendChild(deleteBtn);
    taskList.appendChild(li);
}

// Fungsi untuk menyimpan tugas ke localStorage
function saveTasks() {
    let tasks = [];
    document.querySelectorAll("#task-list li").forEach(function(li) {
        let taskText = li.querySelector("span").textContent;
        let isCompleted = li.querySelector("input[type='checkbox']").checked;
        tasks.push({ text: taskText, completed: isCompleted });
    });
    localStorage.setItem("tasks", JSON.stringify(tasks));
}

// Fungsi untuk memuat tugas dari localStorage
function loadTasks() {
    let savedTasks = localStorage.getItem("tasks");
    if (savedTasks) {
        let tasks = JSON.parse(savedTasks);
        tasks.forEach(function(task) {
            addTask(task.text);
            let lastAdded = taskList.lastChild;
            let checkbox = lastAdded.querySelector("input[type='checkbox']");
            checkbox.checked = task.completed;
            if (task.completed) {
                lastAdded.querySelector("span").style.textDecoration =
"line-through";
            }
        });
    }
}

// Menambahkan elemen ke output
let outputDiv = document.getElementById("output5");
outputDiv.appendChild(inputTask);
outputDiv.appendChild(btnAdd);
outputDiv.appendChild(taskList);

// Memuat tugas yang disimpan
loadTasks();
```

Studi Kasus: Membuat Kalkulator Sederhana

Tugas

Buat sebuah kalkulator sederhana yang dapat melakukan operasi penambahan, pengurangan, perkalian, dan pembagian.

1. **HTML**: Buat antarmuka dengan dua *input field* untuk angka, empat tombol untuk operasi (+, -, *, /), dan satu elemen untuk menampilkan hasil.
2. **JavaScript**: Tulis kode yang mengambil nilai dari kedua *input field* saat sebuah tombol diklik, melakukan perhitungan yang sesuai, dan menampilkan hasilnya pada halaman.

Petunjuk Pengumpulan

1. Buat folder baru di repositori GitLab Anda dengan nama `pertemuan-5`.
2. Buat file `index.html` dan `script.js` di dalam folder ini.
3. Perbarui file `index.html` utama di *root* repositori Anda dengan menambahkan tautan ke tugas ini.
4. *Commit* dan *push* perubahan ke GitLab.

Pertemuan 7

Modul Pertemuan 7: JavaScript dan DOM

Tujuan Pembelajaran

Setelah menyelesaikan modul ini, mahasiswa diharapkan mampu:

- Memahami konsep *Document Object Model* (DOM) dan perannya dalam pengembangan web.
- Mengakses elemen HTML menggunakan JavaScript.
- Memodifikasi konten, atribut, dan *styling* elemen HTML dengan JavaScript.
- Menambahkan dan menghapus elemen HTML secara dinamis.
- Menangani *event* (misalnya: klik, input) pada elemen HTML.

Materi: Interaksi Web dengan JavaScript dan DOM

1. Memahami DOM (*Document Object Model*)

DOM adalah representasi terstruktur dari dokumen HTML sebagai sebuah pohon objek, di mana setiap *node* pada pohon tersebut adalah sebuah elemen HTML. DOM memungkinkan JavaScript untuk mengakses, memanipulasi, dan mengubah struktur serta konten halaman web setelah halaman tersebut dimuat.

2. Mengakses Elemen HTML

Sebelum memanipulasi elemen, kita harus bisa mengaksesnya terlebih dahulu. Berikut adalah beberapa metode yang umum digunakan:

- `document.getElementById('id_elemen')`: Mengakses elemen berdasarkan atribut `id`.
- `document.querySelector('.class_elemen')`: Mengakses elemen pertama yang cocok dengan selector CSS.
- `document.querySelectorAll('.class_elemen')`: Mengakses semua elemen yang cocok dengan selector CSS dalam bentuk *NodeList*.

3. Memanipulasi Konten dan Atribut

Setelah mendapatkan referensi ke sebuah elemen, kita bisa memanipulasinya:

- **Mengubah Konten**:
  - `elemen.innerHTML = 'Konten baru'`: Mengubah konten HTML di dalam elemen.
  - `elemen.textContent = 'Teks baru'`: Mengubah teks di dalam elemen.
- **Mengubah Atribut**:
  - `elemen.setAttribute('href', 'https://example.com')`: Mengatur nilai atribut.
  - `elemen.classList.add('nama-kelas')`: Menambah kelas CSS.

4. Menangani *Events* (Peristiwa)

Event adalah peristiwa yang terjadi di halaman web, seperti klik mouse, penekanan tombol keyboard, atau pengiriman formulir. JavaScript dapat mendeteksi peristiwa ini dan menjalankan kode sebagai responsnya.

- **Metode `addEventListener()`**: Ini adalah metode yang paling direkomendasikan untuk menangani event.

None

-

```
const tombol = document.querySelector('button');

tombol.addEventListener('click', function() {
  alert('Tombol diklik!');
});
```

-

Latihan

Latihan 1: Mengakses Elemen HTML

Gunakan JavaScript untuk:

- Mengakses elemen dengan ID "judul" dan ubah teksnya menjadi "Selamat Datang!"
- Mengakses elemen dengan kelas "deskripsi" dan ubah warnanya menjadi biru
- Mengakses semua elemen <li> dan ubah latar belakangnya menjadi abu-abu muda

HTML

```
<div class="workspace">
                <h1 id="judul">Judul Asli</h1>
                <p class="deskripsi">Ini adalah deskripsi awal.</p>
                <ul>
                    <li>Item 1</li>
                    <li>Item 2</li>
                    <li>Item 3</li>
                </ul>
            </div>
```

JavaScript

```
// Mengakses elemen dengan ID
const judul = document.getElementById('judul');
judul.textContent = 'Selamat Datang!';

// Mengakses elemen dengan kelas
const deskripsi = document.querySelector('.deskripsi');
deskripsi.style.color = 'blue';

// Mengakses semua elemen li
const listItems = document.querySelectorAll('li');
listItems.forEach(item => {
```

```
    item.style.backgroundColor = '#f0f0f0';
});
```

Latihan 2: Memanipulasi Konten dan Atribut

Gunakan JavaScript untuk:

- Mengubah teks dalam elemen dengan ID "teks-ubah"
- Menambahkan kelas "highlight" ke elemen dengan ID "teks-highlight"
- Mengubah atribut href pada link dengan ID "link-ubah"

HTML

```
<div class="workspace">
                <p id="teks-ubah">Teks ini akan diubah.</p>
                <p id="teks-highlight">Teks ini akan disorot.</p>
                <a href="#" id="link-ubah">Link ini akan diubah</a>
            </div>
```

JavaScript

```
// Mengubah teks
const teksUbah = document.getElementById('teks-ubah');
teksUbah.textContent = 'Teks telah berhasil diubah!';

// Menambahkan kelas
const teksHighlight = document.getElementById('teks-highlight');
teksHighlight.classList.add('highlight');
// Tambahkan juga style untuk kelas highlight
const style = document.createElement('style');
style.textContent = '.highlight { background-color: yellow; padding: 5px; }';
document.head.appendChild(style);

// Mengubah atribut href
const linkUbah = document.getElementById('link-ubah');
linkUbah.href = 'https://www.example.com';
linkUbah.textContent = 'Kunjungi Example.com';
```

Latihan 3: Menangani Events (Peristiwa)

Buatlah JavaScript untuk:

- Menambahkan event listener pada tombol dengan ID "tombol-alert" untuk menampilkan alert
- Mengubah warna latar belakang kotak dengan ID "kotak-warna" saat kursor mouse masuk dan keluar
- Menampilkan koordinat mouse saat diklik di dalam div dengan ID "area-klik"

HTML

```
<div class="workspace">
                <div class="interactive-elements">
                    <button id="tombol-alert">Klik untuk Alert</button>
                </div>
                <div id="kotak-warna" class="color-box">Kotak ini akan berubah
warna</div>
                <div id="area-klik" style="height: 100px; border: 1px solid #ccc;
margin: 10px 0; padding: 10px;">
                    Klik di area ini untuk melihat koordinat
                </div>
                <p id="koordinat-tampilan">Koordinat: </p>
            </div>
```

JavaScript

```
// Event listener untuk tombol alert
const tombolAlert = document.getElementById('tombol-alert');
tombolAlert.addEventListener('click', function() {
    alert('Tombol telah diklik!');
});

// Event listener untuk kotak warna
const kotakWarna = document.getElementById('kotak-warna');
kotakWarna.addEventListener('mouseenter', function() {
    this.style.backgroundColor = '#4a6fa5';
    this.style.color = 'white';
});
kotakWarna.addEventListener('mouseleave', function() {
    this.style.backgroundColor = '#ddd';
    this.style.color = 'black';
});

// Event listener untuk area klik
const areaKlik = document.getElementById('area-klik');
const koordinatTampilan = document.getElementById('koordinat-tampilan');
areaKlik.addEventListener('click', function(event) {
    koordinatTampilan.textContent = `Koordinat: X=${event.clientX},
Y=${event.clientY}`;
});
```

Latihan 4: Membuat Elemen Dinamis

Buatlah JavaScript untuk:

- Menambahkan item baru ke dalam list ketika tombol "Tambah Item" diklik
- Menghapus item dari list ketika tombol "Hapus" pada item tersebut diklik
- Mengosongkan seluruh list ketika tombol "Hapus Semua" diklik

HTML

```
<div class="workspace">
                <div class="interactive-elements">
                    <button id="tambah-item">Tambah Item</button>
                    <button id="hapus-semua">Hapus Semua</button>
                </div>
                <ul id="dynamic-list">
                    <li>Item 1 <button class="hapus-item">Hapus</button></li>
                    <li>Item 2 <button class="hapus-item">Hapus</button></li>
                </ul>
            </div>
```

JavaScript

```
// Fungsi untuk menambah item
document.getElementById('tambah-item').addEventListener('click', function() {
    const list = document.getElementById('dynamic-list');
    const itemCount = list.children.length + 1;
    const newItem = document.createElement('li');

    newItem.textContent = `Item ${itemCount} `;

    const deleteButton = document.createElement('button');
    deleteButton.textContent = 'Hapus';
    deleteButton.classList.add('hapus-item');
    deleteButton.addEventListener('click', function() {
        list.removeChild(newItem);
    });

    newItem.appendChild(deleteButton);
    list.appendChild(newItem);
});

// Fungsi untuk menghapus semua item
document.getElementById('hapus-semua').addEventListener('click', function() {
    const list = document.getElementById('dynamic-list');
    list.innerHTML = '';
});

// Menambahkan event listener untuk tombol hapus yang sudah ada
document.querySelectorAll('.hapus-item').forEach(button => {
    button.addEventListener('click', function() {
        this.parentElement.remove();
    });
});
```

Studi Kasus: Membuat Aplikasi To-Do List Interaktif

Tugas

Buat aplikasi *to-do list* sederhana dimana pengguna dapat:

1. **Menambah** tugas baru ke dalam daftar.
2. **Menandai** tugas sebagai selesai (misalnya, dengan mengubah *styling*).
3. **Menghapus** tugas dari daftar.

Petunjuk Praktikum

1. **HTML**: Buat satu *input field* dan satu tombol untuk menambahkan tugas, serta sebuah `<ul>` atau `<div>` kosong untuk menampung daftar tugas.
2. **JavaScript**:
    - Tangani *event* klik pada tombol "Tambah".
    - Ambil nilai dari *input field*.
    - Buat elemen `<li>` baru dan tambahkan ke dalam daftar (*list*).
    - Tambahkan *event listener* pada setiap elemen `<li>` yang baru dibuat, sehingga saat diklik, elemen tersebut bisa ditandai sebagai selesai atau dihapus.

Petunjuk Pengumpulan

1. Buat folder baru di repositori GitLab Anda dengan nama `pertemuan-6`.
2. Buat file `index.html` dan `script.js` di dalam folder ini.
3. Perbarui `index.html` utama di *root* repositori Anda dengan menambahkan tautan ke tugas ini.
4. *Commit* dan *push* perubahan ke GitLab.

Pertemuan 8 - Uji Kompetensi 1

Pertemuan 9

Modul Pertemuan 9: Pengenalan React

Tujuan Pembelajaran

Setelah menyelesaikan modul ini, mahasiswa diharapkan mampu:

- Memahami apa itu *library* JavaScript dan apa fungsi React.
- Mengerti konsep dasar **komponen** dan cara berpikir dengan komponen.
- Memahami sintaks **JSX** dan perbedaannya dengan HTML.
- Membuat dan merender komponen React pertama mereka.

Materi: Membangun dengan Komponen

React adalah sebuah *library* JavaScript untuk membangun antarmuka pengguna. Dibangun dan dikelola oleh Facebook (sekarang Meta), React merevolusi cara *developer* mendekati pengembangan web dengan konsep **komponen**.

1. Mengapa React?

Sebelum React, halaman web dibuat dengan memanipulasi DOM secara langsung. Ini bisa menjadi rumit dan sulit dikelola seiring bertambahnya fungsionalitas. React menyelesaikan masalah ini dengan:

- **Pendekatan Deklaratif**: Anda hanya perlu mendeskripsikan tampilan yang Anda inginkan, dan React akan mengurus sisanya.
- **Reusable Components**: Anda bisa membuat potongan-potongan UI yang dapat digunakan kembali (*reusable*) dan mengelola logikanya secara terpisah.
- **Virtual DOM**: React menggunakan salinan virtual dari DOM yang memungkinkan pembaruan tampilan yang sangat cepat dan efisien.

2. Apa itu Komponen?

Dalam React, antarmuka pengguna dibangun dari potongan-potongan yang disebut **komponen**. Sebuah komponen adalah fungsi JavaScript yang mengembalikan kode tampilan (biasanya JSX).

> Analogi Sederhana:
>
> Bayangkan sebuah halaman web seperti LEGO. Setiap potongan LEGO, seperti blok, roda, atau jendela, adalah sebuah komponen. Anda bisa menggabungkan komponen-komponen ini untuk membangun struktur yang lebih besar. Komponen yang lebih besar ini (misalnya, sebuah mobil) juga dapat dianggap sebagai komponen yang terdiri dari komponen-komponen yang lebih kecil.

Ada dua jenis komponen:

- **Komponen Fungsional**: Ditulis sebagai fungsi JavaScript. Ini adalah cara modern dan yang paling disarankan dalam React.
- **Komponen Berbasis Kelas**: Ditulis sebagai kelas JavaScript. Cara ini masih digunakan dalam kode lama, tetapi umumnya tidak disarankan untuk proyek baru.

3. Memahami JSX

JSX (*JavaScript XML*) adalah ekstensi sintaks yang memungkinkan Anda menulis kode seperti HTML di dalam file JavaScript. Ini membuat kode tampilan lebih mudah dibaca dan dipahami.

- **Perbedaan dengan HTML**:
  - **`className` vs. `class`**: Di JSX, Anda menggunakan `className` alih-alih `class` untuk menambahkan kelas CSS.
  - **Satu *Root Element***: Setiap komponen hanya bisa mengembalikan satu elemen utama.
  - **Kurung Kurawal `{}`**: Anda dapat menyisipkan kode JavaScript, variabel, atau ekspresi di dalam kurung kurawal.

Contoh Penerapan React: Menampilkan Daftar Item

Ada tiga cara untuk menggunakan React, masing-masing dengan keunggulan tersendiri. Untuk pembelajaran, kita akan menggunakan cara paling sederhana, yaitu dengan CDN. Namun, untuk proyek nyata, cara modern adalah standar industri.

1. Cara Sederhana (Menggunakan HTML dan CDN)

Ini adalah cara termudah untuk memulai, tanpa perlu instalasi tambahan. Anda hanya perlu satu file HTML. Contoh di bawah menunjukkan cara menampilkan daftar item menggunakan array di React.

Kode HTML & JavaScript

Salin seluruh kode di bawah ini dan simpan dalam sebuah file bernama `index.html`.

None

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8" />
  <title>Daftar Item React</title>
  <script
src="[https://unpkg.com/react@18/umd/react.development.js](https://
unpkg.com/react@18/umd/react.development.js)" crossorigin></script>
  <script
src="[https://unpkg.com/react-dom@18/umd/react-dom.development.js](
https://unpkg.com/react-dom@18/umd/react-dom.development.js)"
crossorigin></script>
  <script
src="[https://unpkg.com/@babel/standalone/babel.min.js](https://unp
kg.com/@babel/standalone/babel.min.js)"></script>
</head>
<body>
```

```
  <div id="root"></div>

  <script type="text/babel">
    function DaftarBuah() {
      // Data yang akan ditampilkan
      const buah = ['Apel', 'Jeruk', 'Mangga', 'Pisang'];

      return (
        <div>
          <h1>Daftar Buah</h1>
          <ul>
            {/* Menggunakan method .map() untuk membuat elemen list
untuk setiap item */}
            {buah.map((item, index) => (
              <li key={index}>{item}</li>
            ))}
          </ul>
        </div>
      );
    }

    // Mendapatkan elemen root dari HTML
    const root =
ReactDOM.createRoot(document.getElementById('root'));

    // Merender (menampilkan) komponen ke dalam root
    root.render(<DaftarBuah />);
  </script>

</body>
</html>
```

Penjelasan Contoh

- **Komponen `DaftarBuah`**: Ini adalah komponen fungsional yang berisi data (`const buah`) dan logika untuk menampilkan data tersebut.
- **JSX dan `map()`**: Untuk menampilkan setiap item dalam array, kita menggunakan metode `.map()`. Metode ini akan mengulang setiap item di array dan mengembalikan sebuah elemen `<li>` untuk setiap itemnya.
- **`key={index}`**: Setiap elemen yang dibuat secara dinamis dalam list harus memiliki atribut `key` yang unik. Ini membantu React mengidentifikasi dan memperbarui elemen dengan efisien.

- **`ReactDOM.createRoot()` & `root.render()`**: Kode ini bertugas "menghidupkan" aplikasi React kita. `createRoot()` menunjuk ke elemen `<div id="root">` sebagai wadah, dan `render()` menempatkan komponen `DaftarBuah` di dalamnya.

Contoh ini memberikan gambaran yang sangat baik tentang bagaimana React mengelola data dalam bentuk array dan menampilkannya di UI, yang akan menjadi fondasi untuk konsep *state* yang akan kita pelajari di pertemuan berikutnya.

2. Cara Modern (Menggunakan Node.js & Create React App)

Untuk proyek React profesional, kita menggunakan Node.js dan sebuah *bundler* seperti **Create React App (CRA)**. CRA adalah alat yang menyediakan semua konfigurasi yang diperlukan untuk memulai sebuah proyek React modern dengan cepat, tanpa harus mengurus pengaturan yang rumit.

Pendekatan ini menawarkan banyak keunggulan, seperti:

- **Modularity**: Kode dapat dipecah ke dalam banyak file, membuat proyek lebih terorganisir.
- **Development Server**: Server lokal yang sangat cepat untuk pengembangan.
- **Hot Module Reloading**: Perubahan pada kode akan langsung terlihat di browser tanpa perlu me-reload halaman.

Berikut adalah langkah-langkah untuk memulai proyek React modern dengan CRA:

1. **Instalasi Node.js**: Pastikan Anda sudah menginstal Node.js di komputer Anda. Anda bisa mengunduhnya dari situs resmi Node.js. CRA dan alat-alat pengembangannya membutuhkan Node.js.
2. **Buat Proyek Baru**: Buka terminal atau Command Prompt dan jalankan perintah berikut:

```
None
npx create-react-app nama-aplikasi
```

3. Perintah ini akan membuat folder baru bernama `nama-aplikasi` dengan struktur proyek React yang sudah siap pakai.
4. **Masuk ke Folder Proyek**:

```
None
cd nama-aplikasi
```

5. **Jalankan Aplikasi**:

```
None
npm start
```

6. Perintah ini akan menjalankan server lokal, dan aplikasi React Anda akan terbuka di browser secara otomatis.

Setelah menjalankan perintah terakhir, Anda bisa menemukan kode sumbernya di folder `src` dan memodifikasi file `App.js` untuk membuat komponen Anda sendiri. Struktur proyek CRA standar terlihat seperti ini:

```
None

nama-aplikasi/
├── node_modules/
├── public/
├── src/
│   ├── App.css
│   ├── App.js
│   ├── App.test.js
│   ├── index.css
│   ├── index.js
│   └── logo.svg
├── .gitignore
├── package.json
└── README.md
```

3. Alternatif Modern (Menggunakan Node.js & Vite)

Sebagai alternatif modern yang lebih ringan dan cepat dari CRA, Anda bisa menggunakan **Vite**. Vite dirancang untuk mempercepat proses pengembangan, terutama saat proyek menjadi sangat besar.

Berikut adalah langkah-langkah untuk memulai proyek React dengan Vite:

1. **Instalasi Node.js**: Sama seperti CRA, pastikan Node.js sudah terinstal.
2. **Buat Proyek Baru**: Buka terminal atau Command Prompt dan jalankan perintah berikut:

```
None
npm create vite@latest nama-aplikasi -- --template react
```

3. Perintah ini akan membuat folder `nama-aplikasi` dengan struktur proyek React yang minimalis dan teroptimasi.
4. **Masuk ke Folder Proyek**:

```
None
cd nama-aplikasi
```

5. **Instalasi Dependencies**:

None

```
npm install
```

6. **Jalankan Aplikasi**:

None

```
npm run dev
```

7. Aplikasi Anda akan berjalan di alamat `http://localhost:5173` atau port lain.

Contoh-contoh Pembuatan Komponen pada React

Langkah 1: Setup Project dengan Vite + Tailwind

Shell

```
npm create vite@latest belajar-komponen-ui -- --template react
cd belajar-komponen-ui
npm install
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p
```

Langkah 2: Konfigurasi Tailwind

`tailwind.config.js`:

JavaScript

```
/** @type {import('tailwindcss').Config} */
export default {
  content: [
    "./index.html",
    "./src/**/*.{js,ts,jsx,tsx}",
  ],
  theme: {
    extend: {},
  },
  plugins: [],
}
```

`src/index.css`:

CSS

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

Komponen UI Dasar Sederhana

1. Button Component (`src/components/Button.jsx`)

```
None
function Button({ children, onClick, disabled }) {
  return (
    <button
      className="bg-blue-500 text-white px-4 py-2 rounded-lg hover:bg-blue-600
transition-colors"
      onClick={onClick}
      disabled={disabled}
    >
      {children}
    </button>
  )
}

export default Button
```

2. Card Component (`src/components/Card.jsx`)

```
None
function Card({ children, title }) {
  return (
    <div className="bg-white p-6 rounded-lg shadow-md border border-gray-200">
      {title && <h3 className="text-xl font-bold mb-4">{title}</h3>}
      {children}
    </div>
  )
}

export default Card
```

3. Input Field Component (`src/components/InputField.jsx`)

```
None
function InputField({ label, type = 'text', placeholder }) {
  return (
    <div className="mb-4">
      {label && (
        <label className="block text-sm font-medium text-gray-700 mb-1">
          {label}
        </label>
      )}
      <input
        type={type}
        placeholder={placeholder}
```

```
        className="w-full px-3 py-2 border border-gray-300 rounded-lg
focus:outline-none focus:ring-2 focus:ring-blue-500"
      />
    </div>
  )
}

export default InputField
```

4. Alert Component (`src/components/Alert.jsx`)

```
None
function Alert({ children, type = 'info' }) {
  let bgColor = 'bg-blue-100'
  let textColor = 'text-blue-800'

  if (type === 'success') {
    bgColor = 'bg-green-100'
    textColor = 'text-green-800'
  } else if (type === 'warning') {
    bgColor = 'bg-yellow-100'
    textColor = 'text-yellow-800'
  } else if (type === 'error') {
    bgColor = 'bg-red-100'
    textColor = 'text-red-800'
  }

  return (
    <div className={`p-4 rounded-lg ${bgColor} ${textColor} mb-4`}>
      {children}
    </div>
  )
}

export default Alert
```

5. Main App Component (`src/App.jsx`)

```
None
import Button from './components/Button'
import Card from './components/Card'
import InputField from './components/InputField'
import Alert from './components/Alert'

function App() {
  return (
    <div className="min-h-screen bg-gray-100 py-8">
```

```
<div className="max-w-4xl mx-auto px-4">
  <h1 className="text-3xl font-bold text-center text-gray-800 mb-8">
    Belajar Komponen UI React
  </h1>

  {/* Button Examples */}
  <Card title="Contoh Button">
    <div className="space-y-4">
      <Button>Button Biasa</Button>
      <Button disabled>Button Disabled</Button>
    </div>
  </Card>

  {/* Input Field Examples */}
  <Card title="Contoh Input Field" className="mt-6">
    <InputField
      label="Nama Lengkap"
      placeholder="Masukkan nama lengkap"
    />
    <InputField
      label="Email"
      type="email"
      placeholder="email@contoh.com"
    />
    <InputField
      label="Password"
      type="password"
      placeholder="Masukkan password"
    />
  </Card>

  {/* Alert Examples */}
  <Card title="Contoh Alert" className="mt-6">
    <Alert type="info">
      Ini adalah pesan informasi biasa
    </Alert>
    <Alert type="success">
      ✅ Data berhasil disimpan
    </Alert>
    <Alert type="warning">
      ⚠️ Harap periksa kembali data Anda
    </Alert>
    <Alert type="error">
      ❌ Terjadi kesalahan pada sistem
    </Alert>
  </Card>

  {/* Form Example */}
  <Card title="Contoh Form Sederhana" className="mt-6">
    <form className="space-y-4">
```

```
            <InputField
              label="Nama Pengguna"
              placeholder="Masukkan username"
            />
            <InputField
              label="Email"
              type="email"
              placeholder="email@contoh.com"
            />
            <InputField
              label="Password"
              type="password"
              placeholder="Masukkan password"
            />
            <Button>Daftar</Button>
          </form>
        </Card>

        {/* Profile Card Example */}
        <Card title="Kartu Profil" className="mt-6">
          <div className="flex items-center space-x-4">
            <div className="w-16 h-16 bg-blue-500 rounded-full flex items-center
justify-center text-white text-2xl font-bold">
              JS
            </div>
            <div>
              <h4 className="text-lg font-semibold">John Smith</h4>
              <p className="text-gray-600">john.smith@email.com</p>
              <p className="text-gray-500">Web Developer</p>
            </div>
          </div>
          <div className="mt-4">
            <Button>Edit Profil</Button>
          </div>
        </Card>

        {/* Product Card Example */}
        <Card title="Kartu Produk" className="mt-6">
          <div className="bg-gray-200 h-48 rounded-lg mb-4 flex items-center
justify-center">
            <span className="text-gray-500">Gambar Produk</span>
          </div>
          <h4 className="text-xl font-semibold mb-2">Laptop Gaming</h4>
          <p className="text-gray-600 mb-4">Spesifikasi tinggi untuk gaming</p>
          <p className="text-2xl font-bold text-green-600 mb-4">Rp 15.000.000</p>
          <div className="flex space-x-2">
            <Button>Beli Sekarang</Button>
            <button className="px-4 py-2 border border-gray-300 rounded-lg
hover:bg-gray-50">
              Keranjang
```

```
            </button>
          </div>
        </Card>
      </div>
    </div>
  )
}

export default App
```

Penjelasan Komponen:

1. **Button Component**
   - Tombol sederhana dengan style dasar
   - Support disabled state
   - Hover effect
2. **Card Component**
   - Container dengan background putih
   - Shadow dan border
   - Title opsional
3. **InputField Component**
   - Input text dengan label
   - Support berbagai type (text, email, password)
   - Style focus yang jelas
4. **Alert Component**
   - Kotak pesan dengan warna berbeda
   - 4 tipe: info, success, warning, error
   - Simple dan mudah dipahami

Contoh Penggunaan dalam Aplikasi:

Form Sederhana

```
None
<Card title="Form Login">
  <InputField label="Email" type="email" placeholder="Email Anda" />
  <InputField label="Password" type="password" placeholder="Password" />
  <Button>Login</Button>
</Card>
```

Profile Card

```
None
<Card title="Profil Saya">
  <div className="flex items-center">
    <div className="w-12 h-12 bg-blue-500 rounded-full"></div>
    <div className="ml-4">
```

```
      <h4>Nama User</h4>
      <p>user@email.com</p>
    </div>
  </div>
  <Button className="mt-4">Edit Profil</Button>
</Card>
```

Product Display

```
None
<Card>
  <h4>Nama Produk</h4>
  <p>Deskripsi singkat produk</p>
  <p>Rp 100.000</p>
  <Button>Beli</Button>
</Card>
```

Jalankan dengan `npm run dev` untuk melihat hasilnya!

Tugas Praktikum: Membuat Halaman Portofolio Sederhana

Tugas

Buat sebuah halaman portofolio pribadi sederhana menggunakan React dengan pendekatan komponen. Halaman ini harus menampilkan beberapa bagian dasar, seperti perkenalan, daftar proyek, dan kontak.

Petunjuk Praktikum

Gunakan salah satu dari **cara modern** (Create React App atau Vite) yang telah dijelaskan di atas. Ikuti langkah-langkah berikut:

1. **Buat Proyek Baru**: Ikuti instruksi di bagian "Cara Modern" untuk membuat proyek React baru Anda.
2. Buat komponen UI seperti Button, Card, dan komponen lain yang diperlukan dan bisa digunakan secara berulang (reusable).
3. **Buat Komponen-Komponen**: Di dalam folder `src`, buat beberapa komponen fungsional terpisah (misalnya, di file-file terpisah):
    - `Header.jsx` untuk menampilkan nama dan navigasi.
    - `AboutMe.jsx` untuk deskripsi diri singkat.
    - `Projects.jsx` untuk menampilkan daftar proyek.
    - `Contact.jsx` untuk informasi kontak.
4. **Gabungkan Komponen**: Impor semua komponen yang telah Anda buat ke dalam file utama `App.jsx` dan gabungkan mereka untuk membentuk halaman portofolio yang lengkap.

5. **Isi Konten**: Tambahkan konten dummy (teks dan data contoh) untuk mengisi setiap komponen.

Petunjuk Pengumpulan

1. Buat folder baru di repositori GitHub Anda dengan nama `pertemuan-9`.
2. Salin seluruh folder proyek React (`nama-aplikasi`) ke dalam folder `pertemuan-9` tersebut.
3. Perbarui file `index.html` utama di *root* repositori Anda dengan menambahkan tautan ke tugas ini.
4. *Commit* dan *push* perubahan ke GitHub.

Pertemuan 10

Modul Pertemuan 10: State Management dan Hooks

Tujuan Pembelajaran

Setelah menyelesaikan modul ini, mahasiswa diharapkan mampu:

- Memahami konsep **state** di React dan perannya dalam membuat UI yang interaktif.
- Mengenal dan memahami fungsi **React Hooks**, khususnya `useState` dan `useEffect`.
- Mampu mengelola data dinamis dalam komponen.
- Mampu membuat efek samping (*side-effects*) yang terorganisir dalam siklus hidup komponen.

Materi: Mengelola Data dan Efek dengan Hooks

Pada pertemuan sebelumnya, kita telah belajar bahwa komponen adalah "blok bangunan" UI. Namun, agar komponen bisa interaktif dan dinamis, mereka membutuhkan "memori" untuk menyimpan data yang dapat berubah. Inilah peran dari **state**.

1. Apa itu State?

State adalah data internal sebuah komponen yang dapat berubah seiring waktu. Setiap kali state berubah, React akan secara otomatis memperbarui tampilan komponen tersebut.

> Analogi:
>
> Jika komponen adalah sebuah ruangan, maka state adalah barang-barang di dalamnya. Saat Anda memindahkan perabotan atau mengubah dekorasi (mengubah state), tampilan ruangan tersebut akan berubah (komponen di-render ulang).

2. Mengenal React Hooks

Hooks adalah fungsi khusus yang memungkinkan kita "menghubungkan" komponen fungsional dengan fitur-fitur React, seperti state dan siklus hidup komponen. Hooks selalu diawali dengan kata `use`.

Dua hooks paling fundamental adalah `useState` dan `useEffect`.

3. `useState` Hook

Fungsi dari `useState` adalah untuk menambahkan state ke komponen fungsional.

Sintaks Dasar:

```javascript
const [state, setState] = useState(nilaiAwal);
```

- **`state`**: Variabel yang menampung nilai state saat ini.
- **`setState`**: Sebuah fungsi yang digunakan untuk mengubah nilai state. Memanggil fungsi ini akan memicu komponen untuk di-render ulang dengan nilai state yang baru.

- **useState(nilaiAwal)**: Inisialisasi state dengan nilaiAwal yang kita berikan.

4. useEffect Hook

Fungsi dari useEffect adalah untuk menjalankan **efek samping** (*side-effects*) dalam komponen. Efek samping adalah tindakan yang tidak secara langsung mengubah tampilan, seperti:

- Mengambil data dari API.
- Mengatur atau membersihkan timer.
- Menangani langganan (*subscription*).

Sintaks Dasar:

```javascript
useEffect(() => {
  // Kode efek samping
  console.log('Efek dijalankan!');

  return () => {
    // Kode pembersihan (opsional)
    console.log('Efek dibersihkan!');
  };
}, [dependencies]);
```

- **Fungsi Utama**: Kode di dalamnya akan dijalankan oleh React.
- **Array Dependensi ([dependencies])**: Array ini mengontrol kapan efek akan dijalankan kembali.
    - **Array kosong []**: Efek hanya dijalankan **satu kali** saat komponen pertama kali dimuat. Ini ideal untuk mengambil data dari API.
    - **Tidak ada array**: Efek akan dijalankan **setiap kali** komponen di-render ulang.
    - **Berisi variabel/state**: Efek akan dijalankan kembali **hanya ketika** salah satu variabel di dalam array berubah.

Studi Kasus: Aplikasi Penghitung Sederhana (*Counter App*)

Kita akan membuat aplikasi yang menampilkan angka dan dua tombol: satu untuk menambah angka dan satu untuk mereset angka. Kita akan menggunakan useState untuk mengelola angka dan useEffect untuk mengamati perubahannya.

Kode React

Salin kode di bawah ini ke dalam file App.js di proyek React Anda.

```javascript
import React, { useState, useEffect } from 'react';

function App() {
```

```
  // 1. Menggunakan useState untuk mengelola state 'count'
  const [count, setCount] = useState(0);

  // 2. Menggunakan useEffect untuk menjalankan efek samping
  // Efek ini akan dijalankan setiap kali nilai 'count' berubah
  useEffect(() => {
    console.log(`Nilai counter telah diperbarui menjadi:
${count}`);
  }, [count]); // Array dependensi: efek hanya dijalankan saat
'count' berubah

  // Fungsi untuk menambah nilai counter
  const handleIncrement = () => {
    setCount(count + 1);
  };

  // Fungsi untuk mereset nilai counter
  const handleReset = () => {
    setCount(0);
  };

  return (
    <div style={{ textAlign: 'center', marginTop: '50px' }}>
      <h1>Aplikasi Penghitung</h1>
      <h2>{count}</h2>
      <button onClick={handleIncrement} style={{ margin: '5px'
}}>Tambah (+)</button>
      <button onClick={handleReset} style={{ margin: '5px'
}}>Reset</button>
    </div>
  );
}

export default App;
```

Penjelasan Kode:

- Kita mengimpor `useState` dan `useEffect` dari React.
- `useState(0)` membuat state bernama `count` dengan nilai awal `0` dan fungsi pembaruan `setCount`.
- `useEffect` akan mencetak pesan ke konsol setiap kali nilai `count` berubah. Ini membuktikan bahwa efek samping kita berjalan sesuai dengan state.

- onClick pada tombol memanggil fungsi handleIncrement atau handleReset, yang pada gilirannya memanggil setCount untuk memperbarui state.

Tugas Praktikum: Menambahkan Efek Loading pada Halaman Portofolio

Tugas

Tambahkan efek *loading* pada halaman portofolio Anda yang dibuat pada pertemuan 9. Halaman utama (seperti Header, AboutMe, dll.) hanya akan ditampilkan setelah efek *loading* selesai.

Petunjuk Praktikum

1. **Gunakan Proyek dari Pertemuan 9**: Salin seluruh folder proyek React (nama-aplikasi) dari tugas **pertemuan 9** ke dalam folder pertemuan-10 yang baru.
2. **Modifikasi App.js / App.jsx**: Buka file utama komponen Anda (App.js atau App.jsx).
3. **Implementasikan State untuk Loading**: Gunakan useState untuk membuat state isLoading dengan nilai awal true. Contoh:

```
None
const [isLoading, setIsLoading] = useState(true);
```

4. **Simulasikan Efek Loading dengan useEffect**: Gunakan useEffect untuk membuat penundaan. Di dalamnya, gunakan setTimeout untuk mengubah state isLoading menjadi false setelah beberapa detik (misalnya, 2000 milidetik). Pastikan array dependensi adalah [] agar efek ini hanya berjalan sekali saat komponen pertama kali dimuat. Contoh:

```
JavaScript
useEffect(() => {
  setTimeout(() => {
    setIsLoading(false);
  }, 2000); // Tunda selama 2 detik
}, []);
```

5. **Lakukan *Conditional Rendering***: Di dalam return, gunakan if-else atau operator kondisional (? :) untuk menampilkan:
    - Tampilan *loading* (misalnya, Loading... atau pesan apa pun) jika isLoading bernilai true.
    - Seluruh halaman portofolio jika isLoading bernilai false.
6. **Verifikasi Fungsionalitas**: Pastikan:
    - Saat halaman dimuat, hanya pesan "Loading..." yang terlihat.
    - Setelah sekitar 2 detik, halaman portofolio Anda akan muncul secara otomatis.
    - Halaman portofolio Anda tetap berfungsi normal.

Petunjuk Pengumpulan

1. Buat folder baru di repositori GitHub Anda dengan nama pertemuan-10.
2. Salin seluruh folder proyek React dari tugas **Pertemuan 9** ke dalam folder pertemuan-10 tersebut.
3. *Commit* dan *push* perubahan ke GitHub.

Pertemuan 11

Modul Pertemuan 11: React Router & Navigasi

Tujuan Pembelajaran

Setelah menyelesaikan modul ini, mahasiswa diharapkan mampu:

- Memahami konsep *routing* di aplikasi web modern.
- Mengenal dan menginstal pustaka `react-router-dom`.
- Membuat navigasi antar halaman (*link*) di dalam *Single Page Application* (SPA).
- Mampu mengimplementasikan rute dinamis untuk menampilkan detail konten.

Materi: Membangun Aplikasi Satu Halaman (SPA)

Pada arsitektur web tradisional, setiap kali Anda mengklik sebuah tautan, browser akan memuat ulang seluruh halaman dari server. Hal ini membuat pengalaman pengguna terasa lambat.

Sebaliknya, **Single Page Application (SPA)** adalah aplikasi web yang memuat satu halaman HTML tunggal dan memperbarui konten halaman tersebut secara dinamis saat pengguna berinteraksi. Pengalaman pengguna menjadi lebih cepat dan mulus karena tidak ada proses *reload* halaman.

1. Apa itu React Router?

Untuk membangun SPA, kita membutuhkan pustaka yang dapat mengelola rute di sisi klien (*client-side routing*). **React Router** adalah pustaka standar yang paling banyak digunakan untuk tujuan ini. Ia menyinkronkan UI aplikasi Anda dengan URL di peramban.

2. Instalasi

Sebelum memulai, Anda perlu menginstal pustaka `react-router-dom` di proyek React Anda. Buka terminal di folder proyek dan jalankan perintah:

```
None

npm install react-router-dom
```

3. Komponen Utama React Router

React Router menyediakan beberapa komponen penting untuk membangun navigasi:

- **`<BrowserRouter>`**: Komponen utama yang membungkus seluruh aplikasi Anda. Ia menggunakan API History HTML5 untuk menjaga UI tetap sinkron dengan URL.
- **`<Routes>`**: Berfungsi sebagai wadah untuk semua `<Route>` yang Anda definisikan. React Router akan mencari rute yang cocok di dalamnya.
- **`<Route>`**: Mendefinisikan rute individual. Ia memiliki dua atribut utama:
  - `path`: URL yang akan dicocokkan.
  - `element`: Komponen React yang akan dirender saat URL cocok.
- **`<Link>`**: Digunakan sebagai pengganti tag `<a>` untuk navigasi internal antar komponen. Menggunakan `<Link>` akan mencegah *reload* halaman.

Studi Kasus: Membuat Navigasi Sederhana

Kode React

Berikut adalah contoh struktur aplikasi React yang menggunakan `react-router-dom` untuk membuat navigasi sederhana.

Pertama, di file `src/index.js` atau `src/main.jsx`, pastikan Anda telah mengimpor `<BrowserRouter>`.

JavaScript

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import { BrowserRouter } from 'react-router-dom';
import App from './App.jsx';
import './index.css';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </React.StrictMode>,
);
```

Kemudian, di file `src/App.jsx`, kita akan mendefinisikan rute dan komponen navigasi.

JavaScript

```
import { Routes, Route, Link } from 'react-router-dom';

// Komponen Halaman
function Home() {
  return <h1>Halaman Utama</h1>;
}

function Tentang() {
  return <h1>Tentang Saya</h1>;
}

function Kontak() {
  return <h1>Hubungi Kami</h1>;
}
```

```jsx
function App() {
  return (
    <div>
      <nav>
        <ul>
          <li>
            <Link to="/">Beranda</Link>
          </li>
          <li>
            <Link to="/tentang">Tentang</Link>
          </li>
          <li>
            <Link to="/kontak">Kontak</Link>
          </li>
        </ul>
      </nav>

      <hr />

      {/* Definisi Rute */}
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/tentang" element={<Tentang />} />
        <Route path="/kontak" element={<Kontak />} />
      </Routes>
    </div>
  );
}

export default App;
```

Tugas Praktikum: Membangun Aplikasi Portofolio SPA

Tugas

Modifikasi proyek portofolio Anda dari pertemuan sebelumnya untuk mengubahnya menjadi *Single Page Application* (SPA) dengan tiga halaman utama: Beranda, Proyek, dan Kontak.

Petunjuk Praktikum

1. **Gunakan Proyek dari Pertemuan 9**: Salin seluruh folder proyek React (`nama-aplikasi`) dari tugas **pertemuan 9** ke dalam folder `pertemuan-11` yang baru.

2. **Instalasi**: Jalankan `npm install react-router-dom` di terminal dalam folder proyek `pertemuan-11`.
3. **Bungkus Aplikasi**: Di file `src/index.js` atau `src/main.jsx`, impor `BrowserRouter` dan bungkus komponen `<App />` di dalamnya.
4. **Buat Halaman (Komponen)**: Buat tiga komponen terpisah untuk setiap halaman:
    - `src/components/Home.jsx` (berisi konten halaman utama portofolio Anda).
    - `src/components/Projects.jsx` (berisi konten daftar proyek).
    - `src/components/Contact.jsx` (berisi informasi kontak).
5. **Konfigurasi Rute di `App.jsx`**:
    - Impor `Routes` dan `Route` dari `react-router-dom`.
    - Di dalam komponen utama `App`, definisikan `<Routes>` dengan tiga `<Route>` yang mengarah ke komponen halaman yang telah Anda buat.
6. **Tambahkan Navigasi**: Ganti navigasi lama Anda (jika ada) dengan `<Link>` dari `react-router-dom` untuk mengarahkan pengguna ke setiap rute.

Petunjuk Pengumpulan

1. Buat folder baru di repositori GitHub Anda dengan nama `pertemuan-11`.
2. Salin seluruh folder proyek React dari tugas **Pertemuan 9** ke dalam folder `pertemuan-11` tersebut.
3. Perbarui file `index.html` utama di *root* repositori Anda dengan menambahkan tautan ke tugas ini.
4. *Commit* dan *push* perubahan ke GitHub.

Pertemuan 12

Modul Pertemuan 12: Pengenalan Next.js

Tujuan Pembelajaran

Setelah menyelesaikan modul ini, mahasiswa diharapkan mampu:

- Memahami apa itu **Next.js** dan manfaatnya dibandingkan hanya menggunakan React.
- Mengerti konsep **Server-Side Rendering (SSR)** dan **Static Site Generation (SSG)**.
- Mampu membuat proyek Next.js baru dan memahami struktur foldernya.
- Mengimplementasikan sistem *routing* berbasis file untuk membuat beberapa halaman.
- Mampu membuat website statis sederhana dengan navigasi antar halaman.

Materi: Membangun Website dengan Next.js

Pada pertemuan sebelumnya, kita telah belajar bahwa React sangat efektif untuk membangun antarmuka pengguna yang dinamis di sisi klien. Namun, untuk aplikasi web yang lebih besar, ada tantangan seperti performa awal yang lambat dan isu **SEO** (*Search Engine Optimization*) karena konten tidak tersedia saat halaman pertama kali dimuat.

Next.js adalah sebuah *framework* React yang mengatasi tantangan ini. Next.js menyediakan semua fitur yang dibutuhkan untuk membangun aplikasi web modern, termasuk *server-side rendering*, *static site generation*, dan sistem *routing* yang terintegrasi.

1. Mengapa Menggunakan Next.js?

Next.js adalah pilihan populer di industri karena berbagai keunggulannya:

- **Server-Side Rendering (SSR)**: Konten HTML sudah dibuat di server sebelum dikirim ke browser. Ini membuat halaman memuat lebih cepat dan lebih mudah diindeks oleh mesin pencari.
- **Static Site Generation (SSG)**: Next.js dapat membuat file HTML statis dari komponen Anda saat proses *build*. Halaman-halaman ini sangat cepat karena tidak perlu memproses data di server setiap kali diakses.
- **Routing Berbasis File**: Anda tidak perlu menginstal `react-router-dom` atau pustaka serupa. Next.js secara otomatis membuat rute berdasarkan nama file di dalam folder `app/`.
- **API Routes**: Next.js memungkinkan Anda membuat *backend endpoint* langsung di dalam proyek yang sama, mempermudah pengembangan aplikasi *full-stack*.

2. Next.js: React dengan Kekuatan Tambahan

Sangat penting untuk memahami bahwa **Next.js adalah *framework* yang dibangun di atas React.** Ini berarti semua yang telah Anda pelajari tentang React, seperti komponen, JSX, *state* (`useState`), dan *hooks* (`useEffect`), tetap berlaku sepenuhnya di Next.js.

Perbedaan mendasarnya terletak pada cara halaman disajikan kepada pengguna:

Fitur	React Murni (CRA/Vite)	Next.js

Penyajian Halaman	**Client-Side Rendering (CSR)**: Halaman kosong dimuat terlebih dahulu, lalu JavaScript memuat dan merender konten.	**Server-Side Rendering (SSR)** & **Static Site Generation (SSG)**: Konten HTML sudah siap saat halaman dimuat, membuat halaman sangat cepat.
Manajemen Rute	Membutuhkan pustaka tambahan seperti `react-router-dom` dengan konfigurasi manual.	**Built-in & Berbasis File**: Cukup buat file baru di dalam folder `app/` untuk membuat rute.
Performa & SEO	Potensi lambat saat muat awal dan kurang optimal untuk SEO karena konten tidak langsung tersedia.	Lebih cepat dan sangat optimal untuk SEO karena konten sudah ada di server.
API	Membutuhkan server backend terpisah untuk API.	Memiliki **API Routes** bawaan untuk membuat API.

3. Instalasi dan Persiapan Proyek

Untuk memulai proyek Next.js, Anda memerlukan Node.js yang sudah terinstal. Buka terminal atau Command Prompt dan jalankan perintah berikut:

```
None

npx create-next-app@latest nama-aplikasi
```

Setelah menjalankan perintah ini, Anda akan diminta untuk menjawab beberapa pertanyaan konfigurasi, seperti:

- Apakah akan menggunakan TypeScript?
- Apakah akan menggunakan ESLint?
- Apakah akan menggunakan Tailwind CSS?
- Apakah akan menggunakan `src` directory?
- Apakah akan menggunakan App Router? (Pilih: Ya)

Jawablah pertanyaan tersebut sesuai dengan preferensi Anda.

4. Struktur Folder dan Routing

Next.js menggunakan sistem *routing* yang sangat intuitif. Setiap folder dan file di dalam direktori `app` akan secara otomatis menjadi sebuah rute.

Berikut adalah contoh struktur folder yang sederhana:

```
None

nama-aplikasi/
├── app/
│   ├── page.js          // -> Halaman utama (beranda) di URL: /
│   ├── about/
│   │   └── page.js      // -> Halaman "Tentang" di URL: /about
│   ├── blog/
│   │   └── page.js      // -> Halaman "Blog" di URL: /blog
│   └── layout.js        // Layout global untuk semua halaman
├── public/              // Digunakan untuk file statis seperti
gambar
└── package.json
```

Studi Kasus: Membuat Website Statis Sederhana

Berikut adalah contoh cara membuat website statis dengan dua halaman: Beranda dan Tentang. Kita akan menggunakan sistem *routing* berbasis file dan komponen `<Link>` dari Next.js untuk navigasi.

Kode Next.js

```
JavaScript

// File: app/page.js
// Halaman utama (URL: /)

import Link from 'next/link';

export default function HomePage() {
  return (
    <main style={{ padding: '20px', textAlign: 'center' }}>
      <h1>Selamat Datang di Website Saya</h1>
      <p>Ini adalah halaman beranda statis yang dibuat dengan
Next.js.</p>
      <Link href="/about">
        <button style={{ margin: '10px', padding: '10px 20px',
cursor: 'pointer' }}>
          Tentang Saya
        </button>
```

```
      </Link>
    </main>
  );
}

// File: app/about/page.js
// Halaman Tentang (URL: /about)

import Link from 'next/link';

export default function AboutPage() {
  return (
    <main style={{ padding: '20px', textAlign: 'center' }}>
      <h1>Tentang Saya</h1>
      <p>Halo, saya seorang mahasiswa yang sedang belajar Teknologi
Web. Halaman ini adalah contoh aplikasi Next.js statis.</p>
      <Link href="/">
        <button style={{ margin: '10px', padding: '10px 20px',
cursor: 'pointer' }}>
          Kembali ke Beranda
        </button>
      </Link>
    </main>
  );
}
```

Penjelasan Kode:

- Setiap file `page.js` di dalam direktori `app` akan menjadi halaman yang dapat diakses melalui URL.
- Next.js menggunakan komponen **`<Link>`** untuk navigasi. Ini adalah pengganti tag `<a>` yang akan melakukan navigasi tanpa memuat ulang halaman, menjaga pengalaman pengguna tetap mulus.

Tugas Praktikum: Membuat Website Statis dengan Next.js

Tugas

Buat sebuah website statis sederhana menggunakan Next.js dengan dua halaman: Beranda dan Tentang. Gunakan proyek portofolio dari pertemuan sebelumnya sebagai dasar.

Petunjuk Praktikum

1. **Buat Proyek Next.js**: Ikuti instruksi di bagian "Instalasi dan Persiapan Proyek" untuk membuat proyek Next.js baru.

2. **Halaman Utama (/)**: Modifikasi file app/page.js untuk membuat halaman beranda yang berisi perkenalan singkat tentang diri Anda.
3. **Halaman Kedua (/about)**: Buat folder about di dalam direktori app, lalu buat file page.js di dalamnya. Isi halaman ini dengan informasi pribadi, seperti hobi atau riwayat pendidikan.
4. **Navigasi**: Tambahkan navigasi <Link> di kedua halaman agar pengguna dapat berpindah dari Beranda ke Tentang dan sebaliknya.

Petunjuk Pengumpulan

1. Buat folder baru di repositori GitHub Anda dengan nama pertemuan-12.
2. Salin seluruh folder proyek Next.js (nama-aplikasi) ke dalam folder pertemuan-12 tersebut.
3. Perbarui file index.html utama di *root* repositori Anda dengan menambahkan tautan ke tugas ini.
4. *Commit* dan *push* perubahan ke GitHub.

Pertemuan 13

Modul Pertemuan 13: Server Components & Routing

Tujuan Pembelajaran

Setelah menyelesaikan modul ini, mahasiswa diharapkan mampu:

- Memahami perbedaan antara **Server Components** dan **Client Components** di Next.js.
- Mengimplementasikan **rute dinamis** untuk membuat halaman yang fleksibel (misalnya, untuk detail postingan blog).
- Melakukan *data fetching* langsung di dalam Server Component.
- Mampu membangun aplikasi blog sederhana dengan Next.js.

Materi: Server Components & Routing Dinamis

Pada pertemuan sebelumnya, kita telah belajar bahwa Next.js memiliki keunggulan dalam *Server-Side Rendering* (SSR) dan *Static Site Generation* (SSG). Next.js App Router (sejak versi 13) memperkenalkan konsep baru yang mengubah cara kita membangun aplikasi: **Server Components**.

1. Server Components vs. Client Components

Ini adalah konsep paling mendasar di Next.js App Router.

Fitur	Server Components	Client Components
Lokasi Render	Di server	Di browser (klien)
Akses ke API	Ya, bisa langsung mengakses database atau *file system*.	Tidak bisa, harus melalui API *endpoint* atau *client-side fetching*.
Dukungan *Hooks*	Tidak mendukung `useState`, `useEffect`, dll.	Mendukung semua *hooks* React.
Fungsi Utama	Mengambil data dan merender UI statis.	Mengelola interaksi pengguna dan state.

Penanda	Default (tidak perlu penanda khusus).	Harus diawali dengan `'use client';` di bagian atas file.

Kapan Menggunakan Keduanya?

- Gunakan **Server Components** untuk halaman yang kontennya tidak berubah-ubah atau untuk bagian UI yang tidak membutuhkan interaksi (misalnya: blog post, halaman produk, footer). Ini optimal untuk performa dan SEO.
- Gunakan **Client Components** untuk bagian UI yang interaktif (misalnya: tombol "like", formulir, *dropdown* menu).

2. Dynamic Routing (Rute Dinamis)

Dalam membuat aplikasi blog, kita tidak mungkin membuat satu file untuk setiap artikel yang ada. Next.js mengatasi ini dengan **Dynamic Routing**. Anda dapat membuat rute yang fleksibel dengan menggunakan tanda kurung siku `[]` pada nama folder atau file.

Contoh Struktur Folder untuk Blog:

```
None

app/
├── blog/
│   ├── page.js           // Halaman untuk menampilkan daftar semua
artikel blog
│   └── [slug]/
│       └── page.js       // Halaman untuk menampilkan detail satu
artikel
```

``

- `app/blog/page.js` akan merender halaman utama blog (URL: `/blog`).
- `app/blog/[slug]/page.js` akan merender halaman detail untuk URL seperti `/blog/artikel-pertama` atau `/blog/belajar-nextjs`. Bagian `[slug]` akan menjadi parameter yang bisa diakses di dalam komponen.

3. Data Fetching dalam Server Components

Keunggulan utama Server Components adalah kemampuannya untuk mengambil data secara langsung tanpa *hooks* seperti `useEffect`. Next.js memperluas fungsi `fetch` bawaan JavaScript sehingga ia memiliki kemampuan *caching* dan deduplikasi otomatis.

```
None

// Contoh: Mengambil data di Server Component
async function getPost(slug) {
```

```
  // Ini bisa diganti dengan akses database atau fetch dari API
internal
  const posts = [
    { slug: 'blog-satu', title: 'Post Pertama' },
    { slug: 'blog-dua', title: 'Post Kedua' },
  ];
  return posts.find(post => post.slug === slug);
}
```

Studi Kasus: Membangun Aplikasi Blog

Berikut adalah contoh cara membuat aplikasi blog sederhana. Kita akan menggunakan data statis di dalam kode untuk menyimulasikan data dari database.

Kode Next.js

File: app/blog/page.js

JavaScript

```
// File: app/blog/page.js
// Halaman daftar artikel blog

import Link from 'next/link';

// Data statis untuk disimulasikan dari database atau file
eksternal
const posts = [
  { slug: 'belajar-nextjs', title: 'Belajar Next.js', intro:
'Pengenalan dasar Next.js untuk pemula.' },
  { slug: 'react-components', title: 'Memahami Komponen React',
intro: 'Mengenal Server dan Client Components.' },
];

export default function BlogPage() {
  return (
    <main style={{ padding: '20px', maxWidth: '800px', margin: '0
auto' }}>
      <h1>Semua Artikel</h1>
      <ul style={{ listStyle: 'none', padding: 0 }}>
        {posts.map(post => (
```

```
          <li key={post.slug} style={{ marginBottom: '20px',
border: '1px solid #ccc', padding: '15px', borderRadius: '8px' }}>
            <Link href={`/blog/${post.slug}`}>
              <h2 style={{ margin: '0 0 10px 0', color: 'blue'
}}>{post.title}</h2>
              <p style={{ margin: 0 }}>{post.intro}</p>
            </Link>
          </li>
        ))}
      </ul>
    </main>
  );
}
```

File: app/blog/[slug]/page.js

JavaScript

```
// File: app/blog/[slug]/page.js
// Halaman detail artikel blog (rute dinamis)

// Data statis yang sama
const posts = [
  { slug: 'belajar-nextjs', title: 'Belajar Next.js', content:
'Next.js adalah framework React yang powerful. Pelajari cara
menggunakannya...' },
  { slug: 'react-components', title: 'Memahami Komponen React',
content: 'Komponen adalah blok pembangun UI Anda. Ketahui
perbedaannya...' },
];

async function getPostBySlug(slug) {
  return posts.find(post => post.slug === slug);
}

export default async function BlogPostPage({ params }) {
  const post = await getPostBySlug(params.slug);

  if (!post) {
    return (
      <main style={{ padding: '20px', textAlign: 'center' }}>
```

```
        <h1>Artikel Tidak Ditemukan</h1>
      </main>
    );
  }

  return (
    <main style={{ padding: '20px', maxWidth: '800px', margin: '0
auto' }}>
      <Link href="/blog">
        <button style={{ marginBottom: '20px' }}>
          &larr; Kembali ke Blog
        </button>
      </Link>
      <h1>{post.title}</h1>
      <hr />
      <p>{post.content}</p>
    </main>
  );
}
```

Tugas Praktikum: Membangun Aplikasi Blog

Tugas

Modifikasi proyek Next.js Anda dari pertemuan sebelumnya untuk membuat aplikasi blog sederhana. Buatlah:

1. Halaman utama blog (`/blog`) yang menampilkan daftar semua artikel.
2. Halaman detail artikel (`/blog/[slug]`) yang menampilkan konten lengkap.

Petunjuk Praktikum

1. **Gunakan Proyek dari Pertemuan 12**: Lanjutkan dari proyek Next.js yang telah Anda buat pada tugas sebelumnya.
2. **Siapkan Data**: Buatlah sebuah array objek yang berisi data artikel blog (misalnya, `title`, `slug`, dan `content`).
3. **Buat Halaman Daftar Blog**: Buat folder `blog` di dalam direktori `app` dan tambahkan file `page.js` di dalamnya. Tampilkan daftar artikel dari data yang telah Anda buat.
4. **Implementasikan Rute Dinamis**: Buat folder `[slug]` di dalam folder `blog`, dan tambahkan file `page.js` di dalamnya. Di file ini, ambil parameter `slug` dari URL dan tampilkan detail artikel yang sesuai.

Petunjuk Pengumpulan

1. Buat folder baru di repositori GitLab Anda dengan nama `pertemuan-13`.
2. Salin seluruh folder proyek Next.js (`nama-aplikasi`) ke dalam folder `pertemuan-13` tersebut.

3. Perbarui file `index.html` utama di *root* repositori Anda dengan menambahkan tautan ke tugas ini.
4. *Commit* dan *push* perubahan ke GitLab.

Pertemuan 14

Modul Pertemuan 14: Form Handling & Otentikasi

Tujuan Pembelajaran

Setelah menyelesaikan modul ini, mahasiswa diharapkan mampu:

- Mengelola data formulir di sisi klien dengan **React Hooks**.
- Memahami konsep dasar **otentikasi** dan fungsinya.
- Melindungi rute atau halaman tertentu agar hanya bisa diakses oleh pengguna yang sudah terautentikasi.
- Mampu mengimplementasikan sistem otentikasi sederhana di Next.js.

Materi: Form Handling & Proteksi Rute

Pada pertemuan sebelumnya, kita telah membangun aplikasi blog statis. Namun, agar blog tersebut bisa dinamis dan dikelola, kita memerlukan cara untuk berinteraksi dengan pengguna, seperti melalui formulir login atau formulir untuk membuat postingan baru. Kita juga perlu memastikan bahwa tidak sembarang orang bisa mengakses halaman admin.

1. Form Handling di React/Next.js

Mengelola formulir adalah tugas utama dalam pengembangan *front-end*. Di React, state digunakan untuk mengontrol nilai dari elemen formulir.

Pendekatan Sederhana dengan `useState`:

JavaScript

```
'use client';

import { useState } from 'react';
import { useRouter } from 'next/navigation';

export default function LoginForm() {
  const [username, setUsername] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState('');
  const router = useRouter();

  // Data dummy untuk simulasi autentikasi
  const dummyUser = {
    username: 'admin',
    password: 'password123',
  };
```

```
  const handleSubmit = (e) => {
    e.preventDefault();
    setError(''); // Reset pesan error

    if (username === dummyUser.username && password === dummyUser.password) {
      // Jika kredensial benar, simpan status login ke localStorage
      localStorage.setItem('isLoggedIn', 'true');
      console.log('Login berhasil!');
      // Redirect ke halaman yang dilindungi
      router.push('/admin');
    } else {
      // Jika salah, tampilkan pesan error
      setError('Username atau password salah.');
    }
  };

  return (
    <div style={{ padding: '20px', maxWidth: '400px', margin: '0 auto', border: '1px solid #ccc', borderRadius: '8px' }}>
      <h1 style={{ textAlign: 'center' }}>Halaman Login</h1>
      <form onSubmit={handleSubmit} style={{ display: 'flex', flexDirection: 'column', gap: '10px' }}>
        <input
          type="text"
          placeholder="Username"
          value={username}
          onChange={(e) => setUsername(e.target.value)}
          style={{ padding: '10px', borderRadius: '4px', border: '1px solid #ccc' }}
        />
        <input
          type="password"
          placeholder="Password"
          value={password}
          onChange={(e) => setPassword(e.target.value)}
          style={{ padding: '10px', borderRadius: '4px', border: '1px solid #ccc' }}
        />
```

```
        <button type="submit" style={{ padding: '10px',
backgroundColor: '#0070f3', color: 'white', border: 'none',
borderRadius: '4px', cursor: 'pointer' }}>
          Login
        </button>
      </form>
      {error && (
        <p style={{ color: 'red', textAlign: 'center', marginTop:
'10px' }}>{error}</p>
      )}
    </div>
  );
}
```

Karena komponen di atas membutuhkan *state* dan interaksi, ia harus menjadi **Client Component** dengan penanda `'use client';`.

2. Konsep Otentikasi (Authentication)

Otentikasi adalah proses verifikasi identitas pengguna. Ini biasanya dilakukan dengan mencocokkan kredensial (seperti *username* dan kata sandi) yang diberikan pengguna dengan data yang tersimpan di server.

Pada aplikasi Next.js, kita bisa mengimplementasikan logika otentikasi sederhana untuk **melindungi rute (route protection)**, yaitu mencegah akses ke halaman tertentu jika pengguna belum login.

Untuk modul ini, kita akan menggunakan pendekatan simulasi otentikasi, di mana status login disimpan di `localStorage` dan tidak memerlukan *backend* sungguhan.

3. Proteksi Rute Sederhana di Next.js

Kita dapat membuat sebuah *hook* kustom atau sebuah fungsi pembantu untuk memeriksa status login pengguna. Logika ini dapat ditempatkan di dalam komponen yang akan dirender di halaman yang ingin kita lindungi.

JavaScript

```
'use client'; // Pastikan ini Client Component

import { useState, useEffect } from 'react';
import { useRouter } from 'next/navigation';

export default function ProtectedPage() {
```

```
  const [isAuthenticated, setIsAuthenticated] = useState(false);
  const router = useRouter();

  useEffect(() => {
    const userIsLoggedIn = localStorage.getItem('isLoggedIn') ===
'true';
    if (!userIsLoggedIn) {
      router.push('/login'); // Redirect ke halaman login jika
belum autentikasi
    } else {
      setIsAuthenticated(true);
    }
  }, [router]);

  if (!isAuthenticated) {
    return <div>Memeriksa status autentikasi...</div>;
  }

  return (
    <div>
      <h1>Halaman Dilindungi</h1>
      <p>Selamat datang, Anda berhasil login!</p>
    </div>
  );
}
```

Studi Kasus: Melindungi Rute Aplikasi Blog

Kita akan mengimplementasikan sistem otentikasi sederhana pada proyek blog Anda dari pertemuan 13.

Tugas

Modifikasi proyek blog Anda dari pertemuan sebelumnya untuk menambahkan fitur berikut:

1. **Halaman Login**: Buat halaman login (`/login`) dengan formulir yang meminta *username* dan kata sandi.
2. **Halaman Admin (Dilindungi)**: Buat halaman baru (`/dashboard` atau `/admin`) yang hanya bisa diakses setelah pengguna berhasil "login".
3. **Logika Otentikasi**:
    - Saat formulir login dikirim, simpan status `isLoggedIn: 'true'` di `localStorage`.
    - Pada halaman yang dilindungi (`/dashboard`), periksa status di `localStorage`.
    - Jika pengguna belum login, alihkan (redirect) mereka ke halaman login.
4. **Navigasi**: Tambahkan tombol "Login" atau "Logout" yang berubah tergantung status autentikasi pengguna.

Petunjuk Praktikum

1. **Gunakan Proyek dari Pertemuan 13**: Lanjutkan dari proyek Next.js yang telah Anda buat.
2. **Buat Halaman Login**: Buat file `login/page.js` di dalam direktori `app`. Gunakan komponen formulir sederhana yang telah kita pelajari di atas.
3. **Buat Halaman Admin**: Buat file `admin/page.js` dan tambahkan logika proteksi rute di dalamnya.
4. **Simulasi Otentikasi**:
    - Pada komponen formulir login, tambahkan fungsi `handleSubmit` yang akan menyimpan `localStorage.setItem('isLoggedIn', 'true')` setelah formulir dikirim.
    - Tambahkan tombol logout yang akan menghapus item dari `localStorage`.

Petunjuk Pengumpulan

1. Buat folder baru di repositori GitHub Anda dengan nama `pertemuan-14`.
2. Salin seluruh folder proyek Next.js (`nama-aplikasi`) ke dalam folder `pertemuan-14` tersebut.
3. Perbarui file `index.html` utama di *root* repositori Anda dengan menambahkan tautan ke tugas ini.
4. *Commit* dan *push* perubahan ke GitHub.

Pertemuan 15

Modul Pertemuan 15: Menggunakan API Eksternal

Tujuan Pembelajaran

Setelah menyelesaikan modul ini, mahasiswa diharapkan mampu:

- Memahami apa itu **API** (*Application Programming Interface*) dan perannya.
- Memahami konsep **metode HTTP** (`GET`, `POST`, dll.) dan fungsinya.
- Melakukan *data fetching* dari API eksternal menggunakan fungsi `fetch()` di Next.js.
- Mengolah data dalam format **JSON** yang diterima dari API.
- Mampu membangun aplikasi web dinamis yang menampilkan data dari sumber eksternal.

Materi: Menggunakan API Eksternal

Hingga saat ini, data yang kita gunakan dalam proyek masih bersifat statis, yaitu data yang ditulis langsung di dalam kode. Untuk membuat aplikasi yang dinamis dan terhubung dengan dunia nyata, kita perlu berinteraksi dengan **API**.

1. Memahami API

Secara sederhana, API adalah "jembatan" yang memungkinkan dua aplikasi yang berbeda untuk berkomunikasi dan bertukar data.

Ketika Anda membuka aplikasi cuaca, aplikasi tersebut tidak menyimpan semua data cuaca di dalamnya. Sebaliknya, ia mengirimkan permintaan ke API cuaca eksternal dan menerima data cuaca terkini sebagai respons.

Data yang dipertukarkan oleh API umumnya berada dalam format **JSON** (*JavaScript Object Notation*). JSON adalah format teks ringan untuk menyimpan dan mentransfer data. Anda sudah terbiasa dengan objek JavaScript; JSON sangat mirip dengan itu.

Contoh data JSON:

JSON

```
{
  "judul": "Inception",
  "tahun": 2010,
  "sutradara": "Christopher Nolan",
  "aktor": ["Leonardo DiCaprio", "Joseph Gordon-Levitt"]
}
```

2. Memahami Metode HTTP (GET, POST, dll.)

Setiap kali browser atau aplikasi Anda berkomunikasi dengan API, ia menggunakan **HTTP** (***Hypertext Transfer Protocol***) dan mengirimkan permintaan dengan metode tertentu. Metode-metode ini menunjukkan jenis operasi yang ingin Anda lakukan.

Berikut adalah beberapa metode yang paling umum:

- `GET`: Digunakan untuk **mengambil** atau meminta data dari API. Ini adalah metode yang paling sering Anda gunakan untuk menampilkan data (seperti daftar postingan).
- `POST`: Digunakan untuk **mengirimkan** data baru ke API untuk dibuat (misalnya: membuat postingan baru, mengirimkan formulir pendaftaran).
- `PUT` / `PATCH`: Digunakan untuk **memperbarui** data yang sudah ada. `PUT` biasanya digunakan untuk memperbarui seluruh data, sementara `PATCH` untuk memperbarui sebagian data.
- `DELETE`: Digunakan untuk **menghapus** data dari API.

Fungsi `fetch()` secara default menggunakan metode `GET`, tetapi Anda bisa menentukan metode lain dengan menyertakan objek konfigurasi.

3. Data Fetching di Next.js

Karena Next.js mendukung **Server Components**, kita bisa melakukan *data fetching* langsung di server. Ini sangat efisien karena data diambil sebelum halaman dikirim ke browser, menghasilkan waktu muat yang lebih cepat.

Fungsi bawaan JavaScript, yaitu `fetch()`, adalah cara paling umum untuk mengambil data dari API. Next.js telah mengoptimalkan `fetch()` agar bekerja dengan sangat baik di Server Components, termasuk fitur *caching* otomatis.

Contoh Penggunaan `fetch()` dengan Metode `GET`:

JavaScript

```
// Komponen yang mengambil data dari API eksternal
async function getPosts() {
  const res = await
fetch('https://jsonplaceholder.typicode.com/posts');
  // Pastikan respons berhasil (status 200)
  if (!res.ok) {
    throw new Error('Gagal mengambil data.');
  }
  return res.json();
}

export default async function PostsList() {
  const posts = await getPosts();

  return (
    <main style={{ padding: '20px', maxWidth: '800px', margin: '0
auto' }}>
      <h1>Daftar Postingan</h1>
      {posts.slice(0, 10).map(post => (
```

```
        <div key={post.id} style={{ marginBottom: '20px', padding:
'15px', border: '1px solid #ccc', borderRadius: '8px' }}>
          <h3>{post.title}</h3>
          <p>{post.body.substring(0, 100)}...</p>
        </div>
      ))}
    </main>
  );
}
```

Contoh Penggunaan `fetch()` dengan Metode `POST`:

Untuk mengirimkan data, Anda perlu menambahkan `method` dan `body` ke dalam `fetch()`.

JavaScript

```
async function createPost() {
  const newPost = {
    title: 'Judul Postingan Baru',
    body: 'Isi dari postingan baru yang akan dikirim.',
    userId: 1,
  };

  const res = await
fetch('https://jsonplaceholder.typicode.com/posts', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
    },
    body: JSON.stringify(newPost),
  });

  if (!res.ok) {
    throw new Error('Gagal membuat postingan.');
  }

  const result = await res.json();
  console.log('Postingan berhasil dibuat:', result);
}
```

Studi Kasus: Membuat Aplikasi Blog Sederhana

Pada tugas ini, kita akan membangun aplikasi blog sederhana yang mengambil data dari API publik **JSONPlaceholder** dan menampilkannya. Aplikasi akan memiliki dua halaman: daftar postingan dan detail postingan.

Tugas Praktikum

Modifikasi proyek Next.js Anda untuk membuat aplikasi yang menampilkan daftar postingan dari sebuah API publik.

1. **Halaman Daftar Postingan (`/posts`)**: Tampilkan daftar postingan dalam bentuk kartu atau daftar sederhana.
2. **Halaman Detail Postingan (`/posts/[id]`)**: Buat rute dinamis yang menampilkan detail lengkap dari satu postingan.

Petunjuk Praktikum

1. **Gunakan Proyek dari Pertemuan 14**: Lanjutkan dari proyek Next.js yang telah Anda buat.
2. **Pilih API**: Gunakan API publik **JSONPlaceholder** yang tidak memerlukan kunci API. Anda bisa mengambil daftar postingan dari `https://jsonplaceholder.typicode.com/posts` dan detail postingan dari `https://jsonplaceholder.typicode.com/posts/1`.
3. **Fetch Data di Server Component**: Buat Server Component baru, misalnya di `app/posts/page.js`, dan gunakan `async/await` dengan `fetch()` untuk mengambil data daftar postingan.
4. **Implementasikan Dynamic Routing**:
    - Buat folder `[id]` di dalam folder `posts`.
    - Buat file `page.js` di dalam folder `[id]`.
    - Gunakan `params.id` dari URL untuk mengambil data detail postingan dari API yang sesuai, misalnya `https://jsonplaceholder.typicode.com/posts/${params.id}`.
5. **Tampilkan Data**: Render data yang diterima di halaman, pastikan data seperti judul, dan isi postingan ditampilkan dengan benar.

Petunjuk Pengumpulan

1. Buat folder baru di repositori GitLab Anda dengan nama `pertemuan-15`.
2. Salin seluruh folder proyek Next.js (`nama-aplikasi`) ke dalam folder `pertemuan-15` tersebut.
3. Perbarui file `index.html` utama di *root* repositori Anda dengan menambahkan tautan ke tugas ini.
4. *Commit* dan *push* perubahan ke GitLab.

Pertemuan 16

Modul Pertemuan 16: Autentikasi Lanjutan (Auth0)

Tujuan Pembelajaran

Setelah menyelesaikan modul ini, mahasiswa diharapkan mampu:

- Memahami konsep dasar **Identity as a Service (IDaaS)**.
- Mengintegrasikan pustaka Auth0 ke dalam proyek Next.js.
- Mengimplementasikan alur otentikasi (login, logout, dan status pengguna).
- Melindungi rute atau halaman tertentu dengan validasi status otentikasi.

Materi: Integrasi Auth0 untuk Otentikasi

Pada pertemuan sebelumnya, kita telah menggunakan `localStorage` untuk mensimulasikan otentikasi. Pendekatan ini cocok untuk pembelajaran, tetapi dalam aplikasi nyata, kita memerlukan solusi yang lebih aman dan terukur. **Auth0** adalah salah satu penyedia layanan otentikasi terkemuka yang membantu *developer* mengimplementasikan sistem login yang kompleks dengan cepat.

1. Apa itu Auth0?

Auth0 adalah platform Identity as a Service (IDaaS) yang menyediakan semua yang Anda butuhkan untuk otentikasi dan otorisasi, seperti halaman login, pendaftaran, verifikasi email, dan lain-lain.

Dengan Auth0, Anda tidak perlu lagi membangun sistem otentikasi dari nol, sehingga Anda bisa fokus pada fitur utama aplikasi Anda.

2. Konsep Dasar Auth0

- **Tenant**: Lingkungan khusus Anda di Auth0.
- **Application**: Representasi aplikasi Anda di dalam Auth0.
- **Client ID & Client Secret**: Kunci unik untuk mengidentifikasi aplikasi Anda.
- **Domain**: URL unik untuk *tenant* Anda (contoh: `dev-xxxx.auth0.com`).
- **Callback URL**: URL di aplikasi Anda yang akan dituju Auth0 setelah proses login berhasil.

3. Instalasi dan Persiapan

Instalasi pustaka Auth0 untuk Next.js:

```
None

npm install @auth0/nextjs-auth0
```

Selanjutnya, buat file `.env.local` di *root* proyek Anda dan tambahkan konfigurasi Auth0:

None

```
AUTH0_SECRET=PANJANG_RANDOM_STRING_APAPUN
AUTH0_BASE_URL=http://localhost:3000
AUTH0_ISSUER_BASE_URL=https://<DOMAIN_AUTH0_ANDA>
AUTH0_CLIENT_ID=<CLIENT_ID_ANDA>
AUTH0_CLIENT_SECRET=<CLIENT_SECRET_ANDA>
```

4. Mengimplementasikan Alur Otentikasi

a. Membuat Auth0 API Route

Buat file [...auth0].js di dalam folder app/api/auth/. Ini akan secara otomatis membuat endpoint API untuk login, logout, dan otentikasi.

JavaScript

```
// File: app/api/auth/[...auth0]/route.js
import { handleAuth } from '@auth0/nextjs-auth0';

export const GET = handleAuth();
```

b. Membuat Komponen Login/Logout

Buat tombol login di Client Component yang mengarah ke API route yang telah dibuat.

JavaScript

```
// File: app/components/AuthButtons.jsx
'use client';

import { useUser } from '@auth0/nextjs-auth0/client';

export default function AuthButtons() {
  const { user, error, isLoading } = useUser();

  if (isLoading) return <div>Loading...</div>;
  if (error) return <div>{error.message}</div>;

  if (user) {
    return (
```

```
      <div style={{ padding: '10px' }}>
        <p>Welcome, {user.name}!</p>
        <a href="/api/auth/logout">Logout</a>
      </div>
    );
  }

  return <a href="/api/auth/login">Login</a>;
}
```

5. Melindungi Halaman (Proteksi Rute)

Next.js App Router memungkinkan Anda untuk melindungi halaman dengan mudah menggunakan `withPageAuthRequired` dari Auth0.

JavaScript

```
// File: app/dashboard/page.js
// Halaman yang dilindungi

import { getSession } from '@auth0/nextjs-auth0';

// Perhatikan bahwa ini adalah Server Component
export default async function DashboardPage() {
  const { user } = await getSession();

  return (
    <main>
      <h1>Dashboard Pengguna</h1>
      <p>Selamat datang, {user.name}!</p>
      <p>Anda telah berhasil login dan mengakses halaman yang
dilindungi.</p>
    </main>
  );
}
```

Studi Kasus: Melindungi Rute Aplikasi Blog

Kita akan memodifikasi proyek blog yang telah kita buat untuk menambahkan fitur login, logout, dan melindungi halaman admin menggunakan Auth0.

Tugas Praktikum

1. **Gunakan Proyek dari Pertemuan 15**: Lanjutkan dari proyek Next.js Anda.
2. **Daftar ke Auth0**: Buat akun gratis di Auth0 dan buat aplikasi baru (pilih **Regular Web Applications**).
3. **Konfigurasi**: Salin `Domain`, `Client ID`, dan `Client Secret` ke file `.env.local` proyek Anda. Pastikan untuk menambahkan `AUTH0_BASE_URL` dan `AUTH0_SECRET`.
4. **Siapkan Auth0 Wrapper**: Buat file `app/layout.js` untuk membungkus aplikasi Anda dengan `UserProvider` dari Auth0.
5. **Implementasikan Halaman Login/Logout**: Tambahkan tombol login dan logout seperti pada contoh di atas. Tombol ini harus berada di halaman utama atau navigasi.
6. **Buat Halaman yang Dilindungi**: Buat halaman baru, misalnya `/dashboard`, yang hanya bisa diakses oleh pengguna yang sudah terautentikasi. Gunakan contoh di atas untuk mengimplementasikan proteksi rute.

Petunjuk Pengumpulan

1. Buat folder baru di repositori GitLab Anda dengan nama `pertemuan-16`.
2. Salin seluruh folder proyek Next.js (`nama-aplikasi`) ke dalam folder `pertemuan-16` tersebut.
3. Perbarui file `index.html` utama di *root* repositori Anda dengan menambahkan tautan ke tugas ini.
4. *Commit* dan *push* perubahan ke GitLab.

Uji Kompetensi 2

Panduan Proyek Akhir: Aplikasi Next.js

Tujuan Proyek

Proyek akhir ini adalah puncak dari seluruh mata kuliah, di mana Anda akan menggabungkan semua konsep yang telah dipelajari untuk membangun sebuah aplikasi web yang fungsional. Tujuan utamanya adalah untuk menunjukkan pemahaman Anda dalam membuat aplikasi dinamis dengan Next.js, mengelola data, dan mengamankan rute.

Deskripsi Proyek

Anda akan membangun sebuah **aplikasi web full-stack** yang memiliki fitur otentikasi, navigasi dinamis, dan terhubung dengan basis data sungguhan. Anda bisa memilih salah satu ide berikut atau mengajukan ide Anda sendiri:

- **Aplikasi Blog Pribadi**: Menampilkan daftar artikel, halaman detail artikel, dan halaman admin untuk mengelola artikel yang hanya bisa diakses setelah login. Data artikel harus disimpan di database.
- **Daftar Tugas (To-Do List) Pengguna**: Menampilkan daftar tugas untuk pengguna yang sudah login, dengan kemampuan untuk menambah, mengedit, atau menghapus tugas. Data tugas harus disimpan secara persisten untuk setiap pengguna di database.
- **Direktori Film Sederhana**: Menampilkan daftar film (dari API publik), halaman detail untuk setiap film, dan halaman admin yang dilindungi untuk menambahkan film baru ke database Anda.

Fitur dan Konsep Wajib

Proyek Anda **harus** mengimplementasikan semua konsep berikut untuk memenuhi kriteria penilaian:

1. **Next.js sebagai Framework Utama**: Seluruh aplikasi harus dibangun menggunakan Next.js App Router.
2. **Komponen dan Hooks**:
    - Menggunakan komponen fungsional React.
    - Mengelola state menggunakan `useState` (misalnya untuk form handling).
    - Menggunakan `useEffect` untuk efek samping (jika diperlukan pada Client Component).
3. **Routing Dinamis (Route Params)**:
    - Terdapat halaman yang menggunakan rute dinamis, seperti `/blog/[slug]` atau `/produk/[id]`, untuk menampilkan konten unik berdasarkan URL.
4. **Otentikasi & Proteksi Rute**:
    - Implementasi sistem login dan logout menggunakan layanan otentikasi dari platform *serverless* (lihat poin 6).
    - Terdapat minimal satu halaman yang **hanya bisa diakses** jika pengguna sudah terautentikasi (misalnya: `/dashboard` atau `/admin`). Jika pengguna belum login, mereka harus dialihkan ke halaman login.
5. **Data Fetching**:
    - Mengambil data dari **database** yang terintegrasi dengan layanan *serverless* yang Anda gunakan.
    - Data tidak boleh statis atau hanya disimulasikan, tetapi harus dimuat secara dinamis dari database.

6. **Penggunaan Layanan Serverless Backend**:
    - Anda **wajib** menggunakan satu platform *Backend as a Service* (BaaS) seperti **Firebase**, **Supabase**, atau **Nhost** sebagai backend aplikasi Anda.
    - Layanan ini akan digunakan untuk **autentikasi (login/logout)** dan **database (menyimpan dan mengelola data)**.
    - Langkah ini adalah transisi dari proyek-proyek sebelumnya yang menggunakan `localStorage` atau data statis, ke aplikasi *full-stack* yang sesungguhnya.

Petunjuk Pengumpulan

1. Buat folder baru di repositori GitLab Anda dengan nama `proyek-akhir`.
2. Salin seluruh folder proyek Next.js Anda ke dalam folder `proyek-akhir`.
3. Pastikan file `package.json` dan semua dependensi terinstal berada di dalam folder proyek Anda.
4. Perbarui file `index.html` utama di *root* repositori Anda dengan menambahkan tautan ke folder `proyek-akhir`.
5. *Commit* dan *push* seluruh perubahan ke GitLab. Pastikan proyek Anda dapat berjalan dengan baik di lingkungan lokal.

Rubrik Penilaian Proyek Akhir

Rubrik ini memecah setiap kompetensi proyek menjadi aspek-aspek penilaian yang lebih rinci, lengkap dengan deskripsi kriteria dan skala skor untuk evaluasi yang lebih terperinci.

CPMK 01: Kualitas & Struktur Kode

Bobot: 20%

Kompetensi: Kode mudah dibaca, terorganisir, dan memiliki komentar yang jelas. Struktur file dan folder proyek logis dan mudah dinavigasi.

1. Aspek: Kualitas Kode

Deskripsi Kriteria: Penilaian ini berfokus pada kebersihan, keterbacaan, dan efisiensi kode. Ini mencakup penggunaan nama variabel yang jelas, penulisan fungsi yang ringkas, dan penambahan komentar yang memadai untuk menjelaskan logika yang kompleks.

Skor	Penjelasan
4	Kode sangat bersih, terstruktur, dan memiliki komentar yang membantu. Mengikuti praktik terbaik (*best practices*) dan menghindari kode yang berlebihan.
3	Kode bersih dan terkomentasi dengan baik, namun mungkin ada sedikit ruang untuk perbaikan dalam hal efisiensi atau keterbacaan.

2	Kode bisa dibaca, tetapi kurang konsisten dalam gaya atau memiliki komentar yang minim.
1	Kode sulit dibaca, tidak terorganisir, dan tidak memiliki komentar, sehingga sulit dipahami.

2. Aspek: Struktur Proyek

Deskripsi Kriteria: Penilaian ini mengevaluasi organisasi file dan folder proyek. Struktur yang baik memudahkan navigasi, kolaborasi, dan pemeliharaan proyek di masa depan.

Skor	Penjelasan
4	Struktur folder sangat logis, modular, dan sesuai dengan konvensi proyek Next.js. Sangat mudah dinavigasi.
3	Struktur folder cukup logis, tetapi mungkin ada beberapa file yang tidak berada di tempat yang paling optimal.
2	Struktur folder tidak terlalu terorganisir, membuat navigasi sedikit sulit.
1	Struktur folder tidak logis dan acak, menyulitkan pemahaman alur proyek.

CPMK 02, 03, & 04: Penerapan Next.js & React

Bobot: 40%

Kompetensi: Menunjukkan keterampilan dalam penggunaan *framework* pengembangan web modern berbasis JavaScript seperti Angular, React, atau Vue.

1. Aspek: Penerapan Komponen

Deskripsi Kriteria: Proyek menunjukkan pemahaman yang kuat dalam menggunakan fungsionalitas dasar React. Anda berhasil mengelola *state* lokal dengan `useState` atau *library state management* (seperti Context, Zustand, atau NgRx) untuk interaksi pengguna dan menggunakan `useEffect` atau *lifecycle hooks* lainnya untuk menangani efek samping seperti pengambilan data atau manipulasi DOM.

Skor	Penjelasan
4	Mampu menerapkan komponen secara modular dan menggunakan *hooks* atau *lifecycle* yang relevan dengan sangat efektif. Kode sangat terorganisir dan efisien.
3	Mampu menerapkan komponen dan menggunakan *hooks* atau *lifecycle* dengan baik, namun masih ada ruang untuk perbaikan dalam hal efisiensi atau organisasi kode.
2	Menunjukkan pemahaman dasar tentang komponen, tetapi penerapan *hooks* atau *lifecycle* masih kurang tepat atau belum optimal.
1	Penerapan komponen dan *hooks* atau *lifecycle* masih sangat minim dan tidak menunjukkan pemahaman yang cukup.

2. Aspek: Penerapan Routing

Deskripsi Kriteria: Aplikasi menggunakan sistem *routing* yang benar untuk navigasi antar halaman. Rute dinamis (misalnya `/artikel/:id` atau `/produk/:slug`) diimplementasikan dengan tepat dan navigasi berjalan lancar. Terdapat juga proteksi rute dasar untuk halaman yang memerlukan otentikasi.

Skor	Penjelasan
4	Mampu menerapkan *routing* statis dan dinamis dengan sempurna. Proteksi rute terimplementasi dengan logika yang kuat dan efisien.
3	Mampu menerapkan *routing* dengan baik, namun ada beberapa rute dinamis yang belum optimal atau logika proteksi rute masih memiliki celah.
2	*Routing* statis berfungsi, tetapi implementasi *routing* dinamis atau proteksi rute masih belum berjalan dengan baik.

1	Sistem *routing* tidak terimplementasi dengan benar, atau hanya menggunakan *routing* statis dasar tanpa adanya navigasi yang terstruktur.

3. Aspek: Penerapan UI Framework

Deskripsi Kriteria: Desain antarmuka pengguna (UI) aplikasi rapi, fungsional, dan responsif. Penggunaan *framework* UI seperti Tailwind CSS atau library komponen lainnya diterapkan secara konsisten di seluruh aplikasi untuk menciptakan tampilan yang estetis dan profesional. Aplikasi dapat diakses dan digunakan dengan baik di berbagai ukuran layar.

Skor	Penjelasan
4	Desain UI sangat profesional, responsif, dan konsisten. Penggunaan *framework* UI terintegrasi dengan baik dan efisien.
3	Desain UI sudah baik dan responsif, tetapi masih ada ketidakselarasan visual atau penerapan *framework* UI yang kurang konsisten.
2	Aplikasi memiliki tata letak dasar, tetapi tidak sepenuhnya responsif atau penggunaan *framework* UI tidak merata.
1	Tampilan aplikasi tidak terorganisir, tidak responsif, dan tidak menunjukkan penggunaan *framework* UI yang jelas.

CPMK 05: Integrasi Backend & Otentikasi

Bobot: 40%

Kompetensi: Mampu mengintegrasikan layanan *backend as a service* (BaaS) seperti Firebase atau Supabase untuk otentikasi dan operasi *database*.

1. Aspek: Otentikasi

Deskripsi Kriteria: Penilaian ini berfokus pada fungsionalitas sistem otentikasi. Login dan logout harus bekerja dengan benar, dan rute-rute yang dilindungi (protected routes) harus dapat diakses hanya oleh pengguna yang sudah terautentikasi. Pengalihan halaman (redirection) juga harus berfungsi dengan baik.

Skor	Penjelasan

4	Sistem otentikasi berjalan dengan sempurna, termasuk login, logout, dan pengalihan rute yang dilindungi. Sangat aman dan fungsional.
3	Sistem otentikasi berfungsi dengan baik, tetapi mungkin ada kelemahan kecil pada penanganan sesi atau pengalihan rute.
2	Fungsionalitas otentikasi dasar ada, tetapi ada masalah atau celah keamanan yang signifikan (misalnya, rute yang dilindungi masih bisa diakses).
1	Sistem otentikasi tidak berfungsi atau tidak ada sama sekali.

2. Aspek: Operasi CRUD

Deskripsi Kriteria: Penilaian ini mengevaluasi kemampuan aplikasi untuk berinteraksi dengan database. Semua operasi **C**reate, **R**ead, **U**pdate, dan **D**elete harus berfungsi dengan benar untuk mengelola data secara dinamis.

Skor	**Penjelasan**
4	Semua operasi CRUD (buat, baca, perbarui, hapus) diimplementasikan dengan sempurna dan berjalan tanpa masalah.
3	Operasi CRUD sebagian besar berfungsi, tetapi ada satu atau dua operasi yang belum optimal atau memiliki *bug* kecil.
2	Hanya beberapa operasi CRUD yang berfungsi, atau ada *bug* besar yang mengganggu fungsionalitas.
1	Operasi CRUD tidak terimplementasi atau tidak berfungsi.

3. Aspek: Pengambilan Data

Deskripsi Kriteria: Penilaian ini mengevaluasi bagaimana aplikasi mengambil dan menampilkan data dari backend. Data harus diambil secara dinamis dari database dan ditampilkan dengan benar di antarmuka pengguna.

Skor	Penjelasan
4	Pengambilan data dari database sangat efisien dan data ditampilkan secara dinamis dan akurat di antarmuka pengguna.
3	Pengambilan data berfungsi dengan baik, tetapi ada sedikit keterlambatan atau data tidak selalu diperbarui secara *real-time* jika diperlukan.
2	Pengambilan data ada, tetapi tidak stabil atau ada masalah dalam menampilkan data yang diambil.
1	Aplikasi tidak dapat mengambil data secara dinamis dari database.

Worksheet 4.1

Worksheet Laporan Praktikum Pertemuan 4: CSS Lanjutan

Nama Mahasiswa	
NIM	
Tanggal Pengumpulan	

1. Tautan Tugas

Deskripsi	**Tautan**
Repositori GitHub/GitLab (Githun repo)	[Tempel Tautan Repositori Anda di sini]
Laman Web Aktif (Github page)	[Tempel Tautan GitHub Pages Anda di sini]

2. Implementasi CSS Grid (Layout Utama)

Tuliskan implementasi Anda terhadap CSS Grid untuk struktur utama halaman (Header, Konten Utama, Footer).

Komponen	**Properti CSS yang Digunakan**	**Tujuan Penerapan**
Kontainer Utama (Body/App)	display: grid;	[Jelaskan: Untuk mengaktifkan layout dua dimensi]
Pembagian Kolom Desktop	grid-template-columns: ...;	[Jelaskan: Bagaimana Anda membagi area Sidebar dan Konten Utama (misalnya: 200px 1fr)]
Penempatan Elemen	grid-area: ...; (atau sejenisnya)	[Jelaskan: Bagaimana Anda memastikan Header dan Footer memanjang penuh]

Hasil Layout Desktop (Grid)

Jelaskan: Bagaimana tampilan layout utama Anda (Header, Sidebar, Konten Utama, Footer) di layar desktop (lebar > 768px)?

3. Implementasi Flexbox (Layout Komponen)

Tuliskan implementasi Anda terhadap Flexbox pada **dua komponen wajib** (Navigasi dan Kartu/Galeri).

Komponen	**Properti CSS yang Digunakan**	**Tujuan Penerapan**
Menu Navigasi	`display: flex; justify-content: ...;`	[Jelaskan: Cara Anda menyejajarkan logo di kiri dan menu di kanan]
Kartu Portofolio/Galeri	`display: flex; flex-wrap: wrap;`	[Jelaskan: Cara Anda menyusun kartu/item secara horizontal]

4. Implementasi Media Queries (Responsivitas)

Jelaskan bagaimana Anda menggunakan Media Queries untuk memastikan layout beradaptasi pada tampilan mobile.

Target Tampilan	**Kode Media Query**	**Perubahan Layout yang Diimplementasikan**
Tampilan Mobile	`@media (max-width: 768px) { ... }`	[Jelaskan: Perubahan Grid Utama menjadi 1 kolom vertikal]

Navigasi Mobile	[Sebutkan selector]	[Jelaskan: Perubahan Flexbox Navigasi menjadi vertikal (flex-direction: column)]
Kartu/Galeri Mobile	[Sebutkan selector]	[Jelaskan: Perubahan Flexbox Kartu/Galeri menjadi vertikal (flex-direction: column)]

Hasil Layout Mobile

Jelaskan: Bagaimana tampilan website Anda di layar mobile (lebar <= 768px)? Apakah semua elemen tersusun vertikal?

5. Refleksi

Apa tantangan terbesar yang Anda hadapi saat mengimplementasikan CSS Grid atau Flexbox, dan bagaimana Anda menyelesaikannya?

Properti CSS mana yang paling efektif Anda gunakan untuk memastikan responsivitas website, dan mengapa?

Sebutkan satu hal baru yang Anda pelajari dari tugas praktikum ini.